

Last Lecture Review

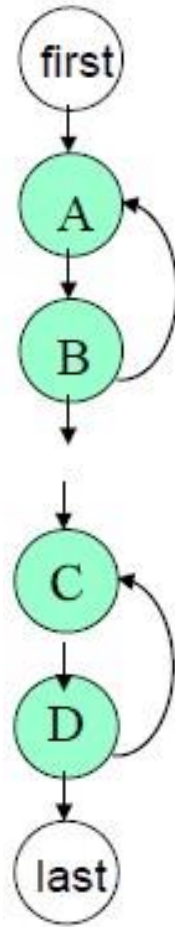
- White Box Testing
- Control Flow Testing
 - Statement/Line/Basic block/Segment Coverage
 - Decision (Branch) Coverage
 - Condition Coverage
 - Multiple Condition Coverage
 - Decision/Condition Coverage
 - Loop Coverage

What Will You Learn Today?

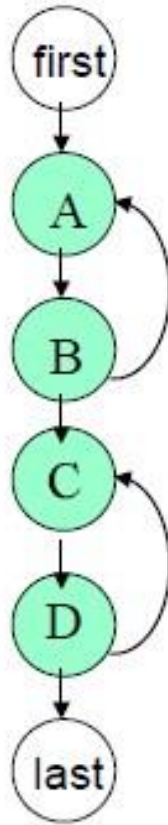
- Loop Coverage
- Path Coverage
- Cyclomatic Complexity
- McCabe's Baseline Method

White Box Testing

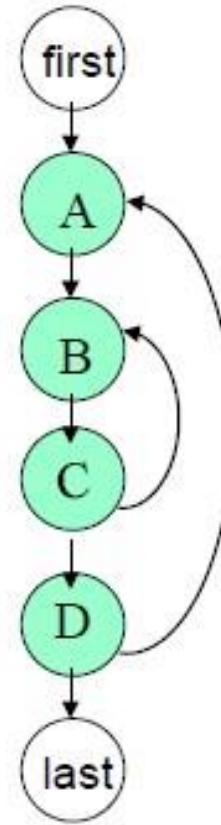
Loop Coverage



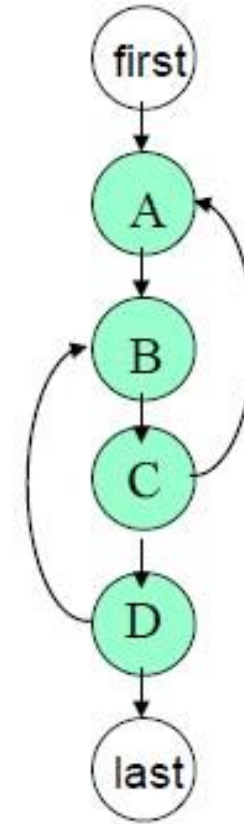
Simple



Concatenated



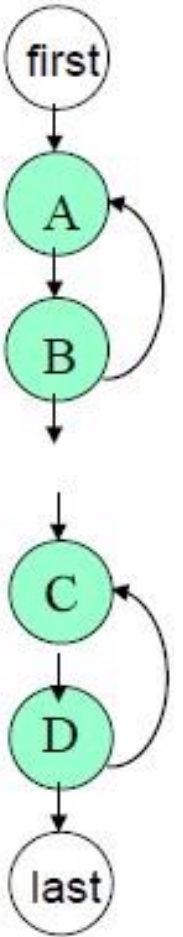
Nested



Unstructured

Loop Coverage

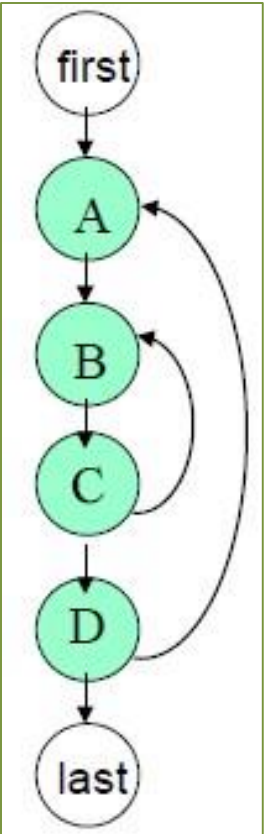
- **Simple loops :**
 - Skip the loop entirely
 - Only one pass through the loop
 - Two passes through the loop
 - m passes through the loop when $m < n$
 m : small number representing a typical loop value
 n : maximum number of allowable passes through the loop
 - $n-1, n, n+1$ passes through the loop



Loop Coverage

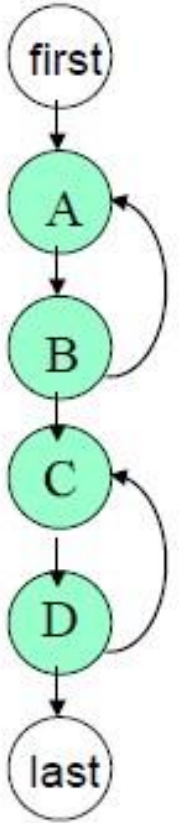
- **Nested loops:**

- Start at the **innermost loop**, set all other loops to **minimum value**.
- Conduct Simple **loop test** for **innermost loop** Add other tests for out-of-range or excluded values.
- Continue until all loops have been tested.



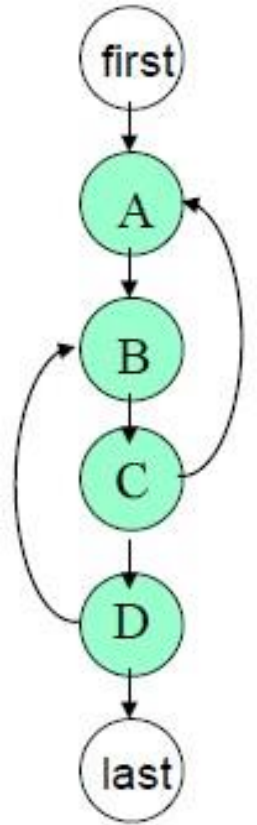
Loop Coverage

- **Concatenated loops:**
 - If each of the **loops** is **independent** of the other, use **Simple loops approach**
 - If the **loops** are **concatenated** and **dependent** (counter for loop 1 is used as the initial value for loop 2) then use **Nested loop approach**.



Loop Coverage

- **Unstructured loops:**
 - Whenever possible, this class of loops should be **redesigned** to reflect the use of the structured programming constructs.



Path Coverage

- A **path** is a sequence of **branches**, or **conditions**
- A path corresponds to a **test case**, or a **set of inputs**
- In **code coverage testing**, **branches** have **more importance** than the **blocks** they connect
- **Bugs** are often **sensitive** to **branches** and **conditions**

Independent paths

- An independent path is any path through the program that introduces at least one new set of processing statements or a new condition
- When stated in terms of a flow graph, an independent path must move along at least one edge that has not been traversed before the path is defined

What is Cyclomatic Complexity?

Cyclomatic complexity is a source code complexity measurement that is being correlated to a number of coding errors. It is calculated by developing a Control Flow Graph of the code that measures the number of linearly-independent paths through a program module.

Lower the Program's cyclomatic complexity, lower the risk to modify and easier to understand. It can be represented using the below formula:

Cyclomatic Complexity

- Cyclomatic Complexity is a **software metric** that provides a **Quantitative measure** of the **logical complexity** of a program
- When used in context of the basis path testing, the **number of independent paths** computed for cyclomatic complexity defines the **number of independent paths** in the basis set of a program and provides us with an **upper bound** for the **number of tests** that must be conducted to ensure that all statements have been executed at least once

Mathematical values involves

Computation of Cyclomatic Complexity

- Cyclomatic complexity has a foundation in graph theory and is computed in the following ways:
- Cyclomatic complexity $V(G)$, for a flow graph, G , is defined as:

$$V(G) = E - N + 2P$$

E: No. of edges

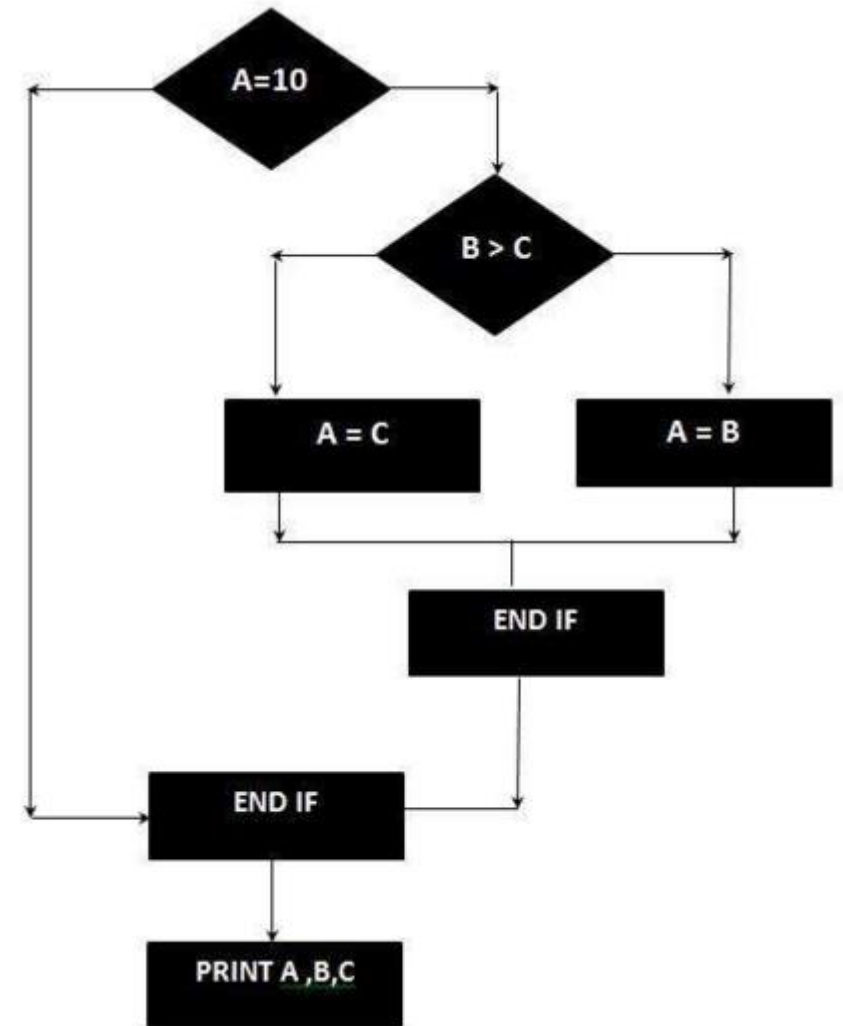
N: No. of nodes

P: No. of disconnected parts of the graph

```
IF A = 10 THEN
  IF B > C THEN
    A = B
  ELSE
    A = C
  ENDIF
ENDIF
Print A Print B Print C
```

$$V(G) = E - N + 2P$$

$$8 - 7 + 2 = 3$$



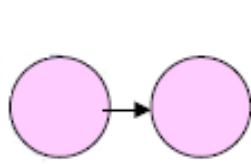
Computation of Cyclomatic Complexity

- Cyclomatic complexity has a foundation in **graph theory** and is computed in the following ways:
- Cyclomatic complexity $V(G)$, for a flow graph, G , is defined as:

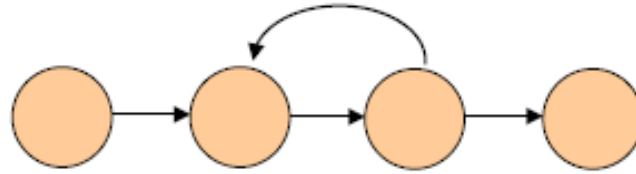
$$V(G) = b + 1$$

b: number of binary decision

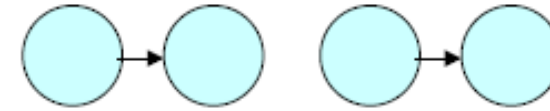
Computation of Cyclomatic Complexity



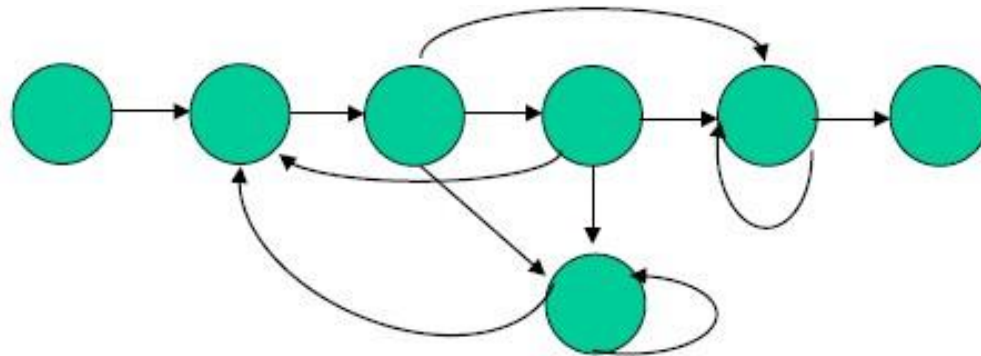
$$E=, N=, P=$$
$$V=1-2+2=1$$



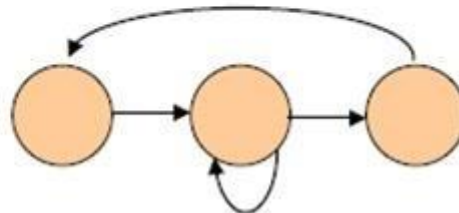
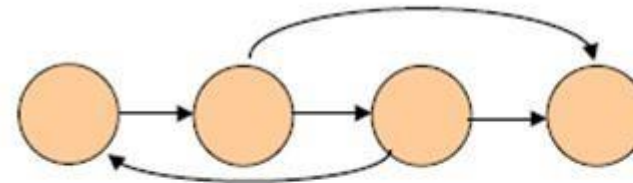
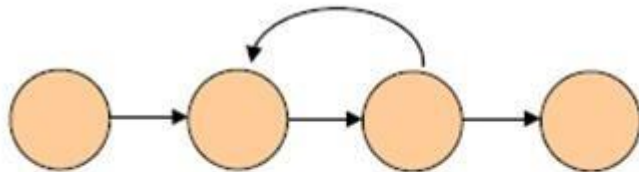
$$E=, N=, P=$$
$$V=4-4+2=2$$



$$E=, N=, P=$$
$$V=2-4+2*2=2$$

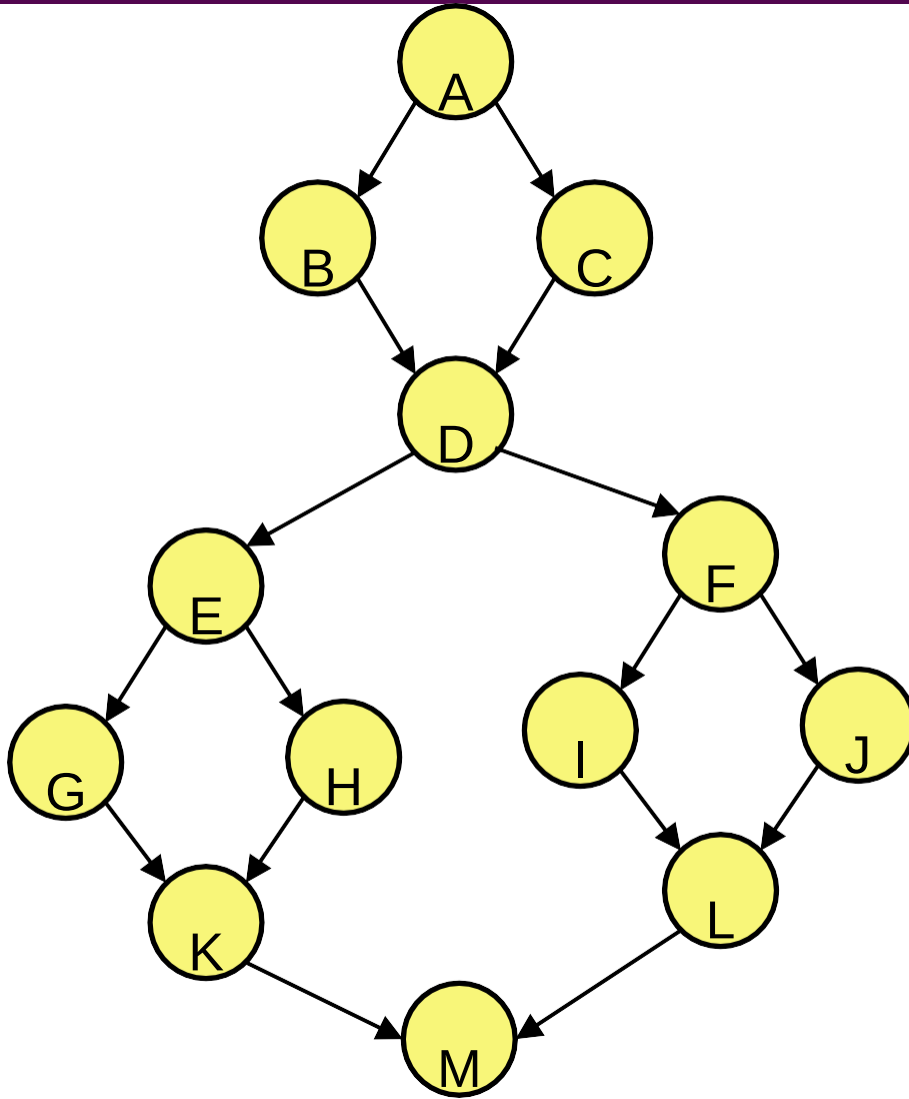


$$E=, N=, P=$$
$$V=12-7+2=7$$



$$E=, N=, P=$$
$$V=13-11+2*3=8$$

Computation of Cyclomatic Complexity



$$V(G) = E - N + 2P$$

$$E = ?$$

$$N = ?$$

$$P = ?$$

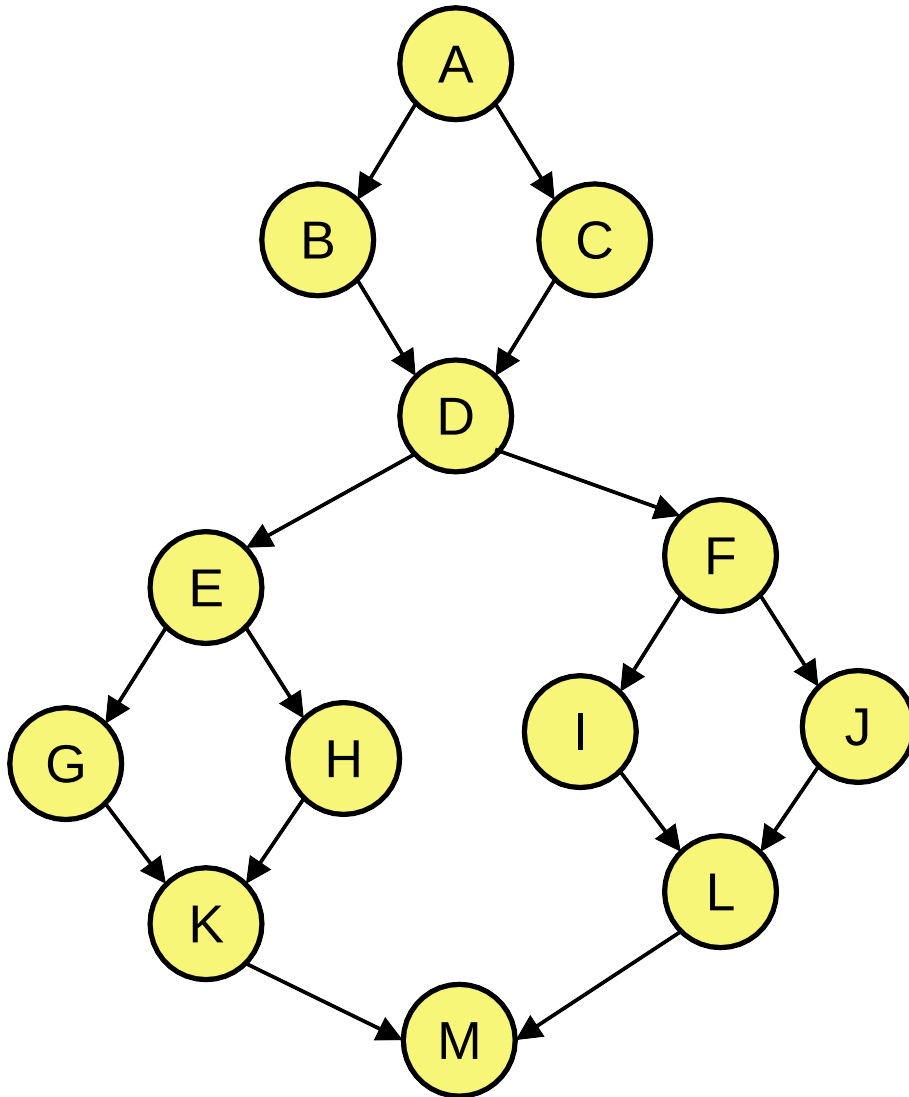
$$V(G) = ?$$

$$V(G) = b + 1$$

$$b = ?$$

$$V(G) = ?$$

Computation of Cyclomatic Complexity



$$V(G) = E - N + 2P$$

$$E = 16$$

$$N = 13$$

$$P = 1$$

$$V(G) = 5$$

$$V(G) = b + 1$$

$$b = 4$$

$$V(G) = ?$$

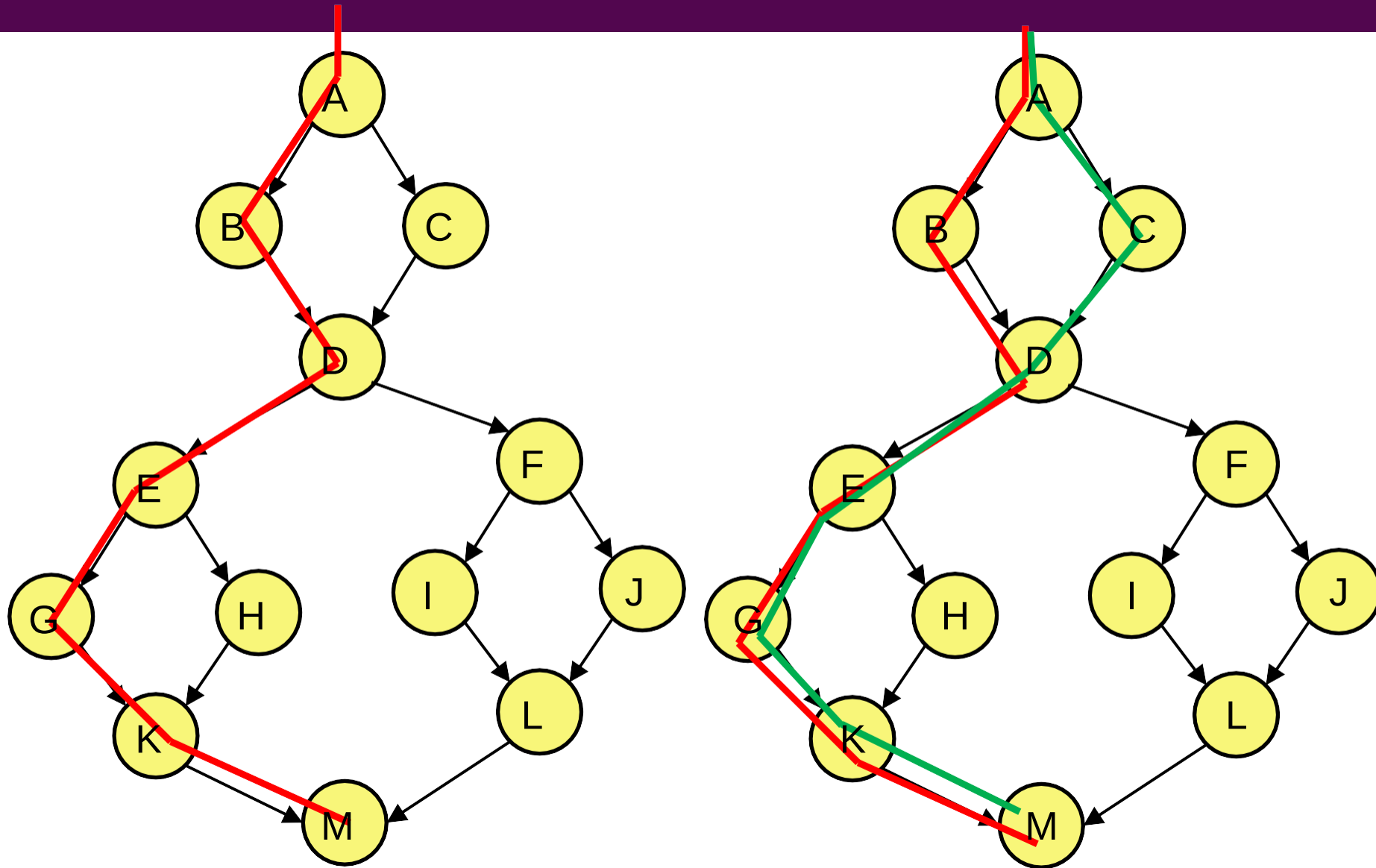
Basis Path Testing

- Derive the **control flow graph**
- **Compute** the graph's **Cyclomatic Complexity** of the resultant flow graph
- Determine a **basis set** of linearly **independent paths**
- Create a **test case** for **each basis path**
- Execute these tests

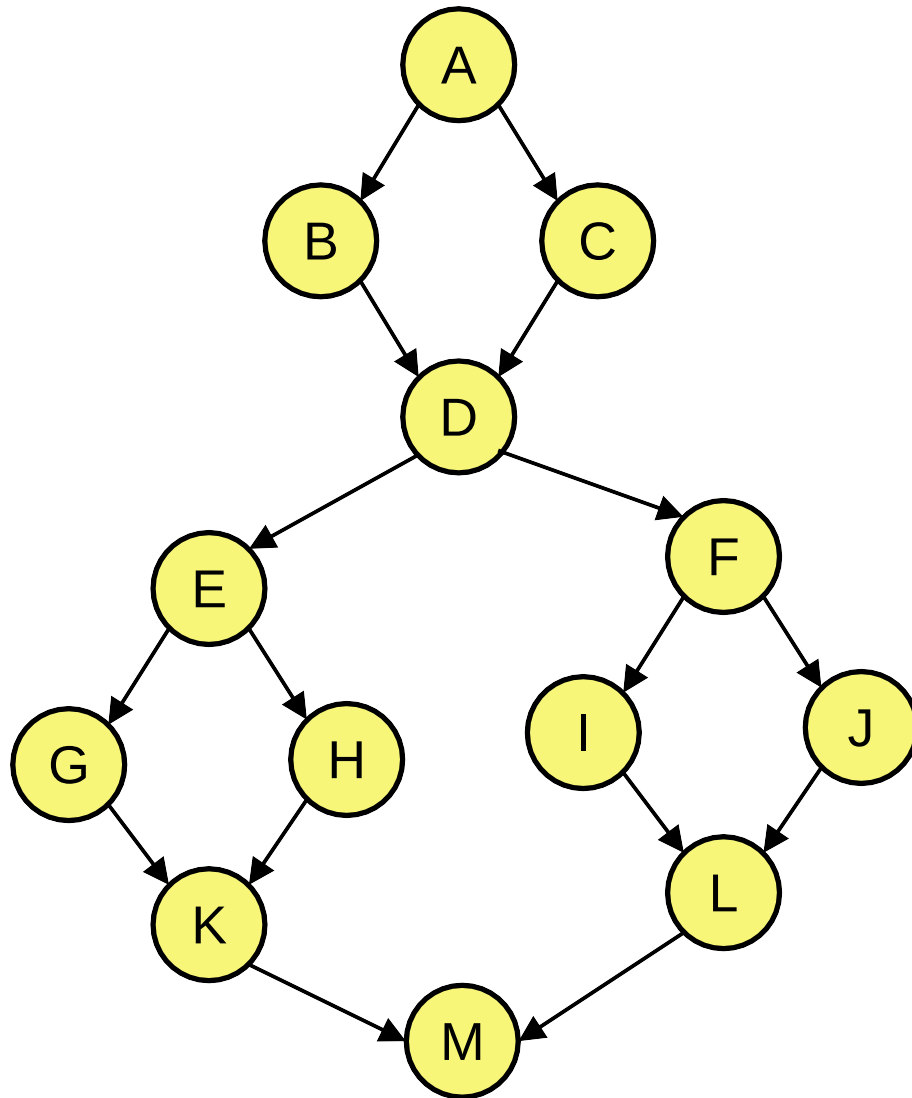
McCabe's Baseline Method

- Pick a “**baseline**” path. This path should be a “**normal case**” program execution. **McCabe advises:** choose a path with as many decision as possible.
- To choose the **next path**, change the outcome of the first decision along the baseline path while keeping the maximum number of other decisions the same as the baseline path.
- To generate the **third path**, begin again with the baseline but vary the **second decision** rather than the **first**.
- Repeat the **3** for other paths until all decision along baseline path have been flipped.
- Now proceed to the **second path**, flipping its decisions, one by one until the basis path set is completed.

McCabe's Baseline Method



McCabe's Baseline Method



Set of basis paths:

1. A ?
2. A ?
3. A ?
4. A ?
5. A ?

Observation

- Basis path testing calls for the **creation** of a **test case** for each of these **paths**
- This set of test cases will **guarantee** both **statement** and **branch** coverage
- Note that multiple sets of basis paths can be created that are not necessarily unique. Each set, however, has the **property** that a **set of test cases** based on it will **execute every statement** and every **branch**

Applicability and Limitation

- + **Control flow testing** is the **basis** of **unit testing**
- + It should be used for all **modules** of code that **cannot be tested sufficiently** through reviews and inspections

Applicability and Limitation

- Its limitation are that the **tester** must have sufficient **programming skill** to understand the code and its control flow
- Control flow testing can be **very time consuming** because of all modules and basic paths that comprise a system

Task

```
a = 1;
```

```
b = 2;
```

```
c = 3;
```

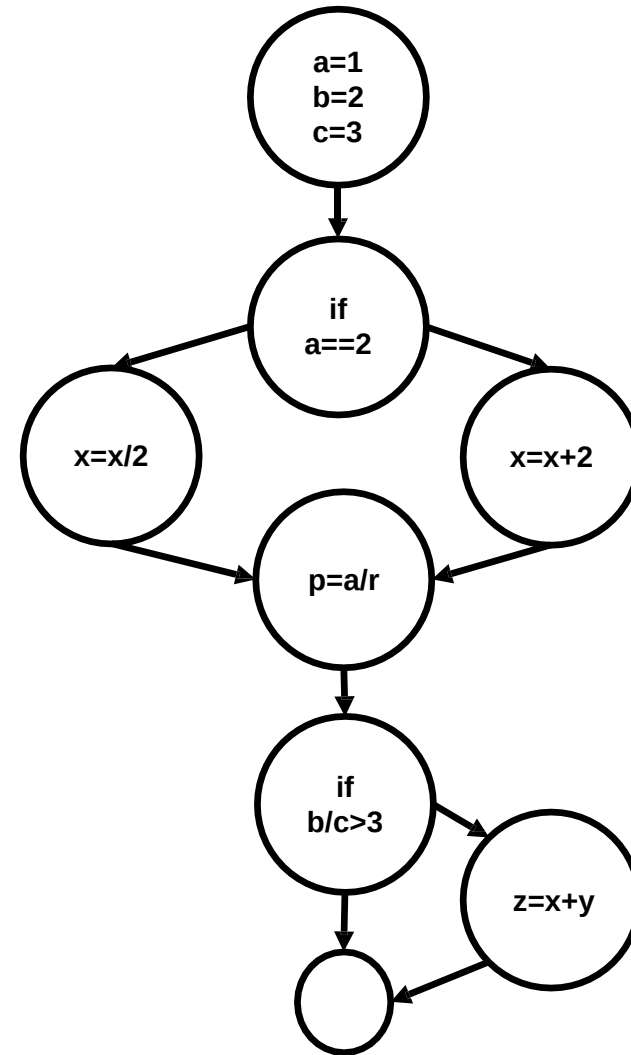
```
if (a==2) {x = x / 2;}
```

```
else{x = x + 2;}
```

```
p = a / r;
```

```
if (b / c > 3)
```

```
{z= x + y;}
```



Data Flow Testing

Data Flow Testing

White box testing

- Control Flow Testing
- Data Flow Testing
 - Define / Use Testing
 - Slice-Based Testing

Data Flow Testing

- Data flow testing **focuses** on the points at which **variables** **receive values** and the points at which these **values are used** (or referenced)
- It **detects improper use of data values** (data flow anomalies) due to coding errors

Data Flow Testing

Static (compile time, inspection)

- Static analysis techniques

Dynamic (execution time) variants

- Look at coverage of data flow properties

MS Level

Data Flow Testing - Static Variants

- Early data flow analyses often centered on a **set of faults** that are known as **define/reference anomalies**
 - A variable that is **defined but never used** (referenced)
 - A variable that is **used but never defined**
 - A variable that is **defined twice before it is used**

Kill and again kill

Main Categories Of Data Flow Coverage

- **Basic blocks** (segment of code without branching)
- **all-use**
- **c-use** (computation use)
- **p-use** (predicate use)
- **all-def**
- **du-path** (definition-use-path)

Basic blocks, defs and uses

- Identify the basic blocks
- Identify all the **definitions**
 - Where variables get their values
- Identify all the **uses**
 - Where variables are used
 - Indicate **p-uses** (in predicates)
 - Indicate **c-uses** (in computations)
- Draw path from each definition to each use that might get data from that definition

```
1.  s = 0;  
2.  i = 1;  
3.  while (i <= n)  
    {  
4.      s + = i;  
5.      i ++  
    }  
6.  cout << s;  
7.  cout << i;  
8.  cout << n;
```

Basic blocks, defs and uses

- Identify the basic blocks
- Identify all the definitions

Basic block: a part of code that executes without branching

- Where variables are used
- Indicate p-uses (in predicates)
- Indicate c-uses (in computations)
- Draw path from each definition to each use that might get data from that definition

```
1. s = 0;  
2. i = 1;  
3. while (i <= n)  
   {  
4.     s += i;  
5.     i ++  
   }  
6. cout << s;  
7. cout << i;  
8. cout << n;
```

Basic blocks, defs and uses

- Identify the basic blocks
- Identify all the definitions
 - Where variables get their values
- Identify all the uses
 - Where variables are used
 - Indicate p-uses (in predicates)
 - Indicate c-uses (in computations)
- Draw path from each definition to each use that might get data from that definition

```
1. s = 0;      Def(s)
2. i = 1;      Def(i)
3. while (i <= n)
   {
4.     s += i;  Def(s)
5.     i++;     Def(i)
   }
6. cout << s;
7. cout << i;
8. cout << n;
```

Basic blocks, defs and uses

p-use: a path starts from a definition of the variable and ends in a statement in which it appears inside a certain predicate

- Identify all the uses
- Where variables are used
- Indicate p-uses (in predicates)
- Indicate c-uses (in computations)

c-use: a path starts from a definition of the variable and ends at a statement in which it is involved in computation

```
1.  s = 0;
2.  i = 1;
3.  while (i <= n)
    {
4.      s += i;
5.      i ++
    }
6.  cout << s;
7.  cout << i;
8.  cout << n;
```

p-use(i),
p-use(n)

c-use(i)

c-use(i)

c-use(s)

c-use(i)

c-use(n)

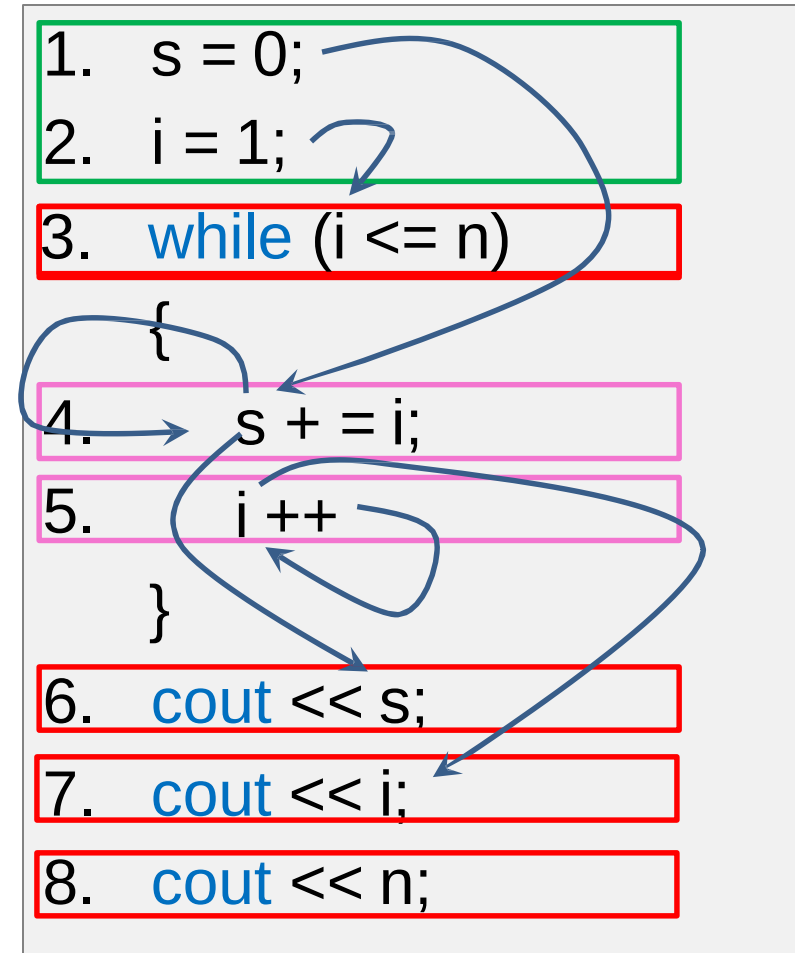
Basic blocks, defs and uses

- Identify the basic blocks
- Identify all the definitions
 - Where variables get their values
- Identify all the uses
 - Where variables are used
 - Indicate p-uses (in predicates)
 - Indicate c-uses (in computations)
- Draw path from each definition to each use that might get data from that definition

```
1.  s = 0;  
2.  i = 1;  
3.  while (i <= n)  
    {  
4.      s += i;  
5.      i ++  
    }  
6.  cout << s;  
7.  cout << i;  
8.  cout << n;
```

Basic blocks, defs and uses

- Identify the basic blocks
- Identify all the definitions
 - Where variables get their values
- Identify all the uses
 - Where variables are used
 - Indicate p-uses (in predicates)
 - Indicate c-uses (in computations)
- Draw path from each definition to each use that might get data from that definition



Data Flow Graph $G(P) = (N, E)$

- Given a program (P) written in an imperative programming language, its **program graph (G)** is a **directed graph** in which **nodes(N)** are **statement** fragments (basic block), and **edges (E)** represent **flow of control**. In addition it details the **definition**, **use** and **destruction** of each of the module's variable.

Notations / Symbols

- **d:** defined, created, initialized, etc.
- **k:** killed, undefined, released
- **u:** used for something
 - **c:** used in a computation
 - **p:** used in a predicate

Notations / Symbols

- $\sim$: Leading “ $\sim$ ” symbol means that nothing of interest (d, k, u) occurs prior to the action noted along the entry-exit path of interest

$\sim d$	the variable does not exist , then it is defined	Ok
$\sim u$	the variable does not exist , then it is used	Possibly anomalous
$\sim k$	the variable does not exist , then it is killed	Possibly anomalous

Notations / Symbols

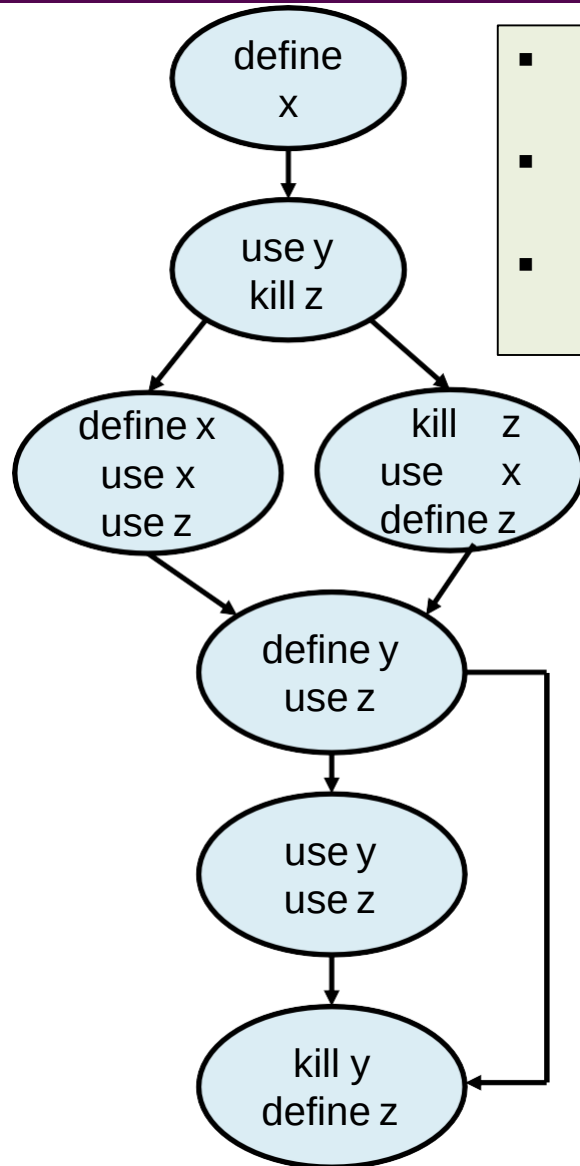
- **~** : Trailing “~” symbol means that nothing happens after the point of interest to the exit

d~	the variable was defined but not used on this path	Possibly anomalous
u~	the variable was used but not killed on this path	Not anomalous
k~	the last thing done on this path was to kill the variable	Not anomalous

Possible Combinations For D, K And U

dd	defined and defined again	Not invalid but suspicious
du	defined and used	Perfectly correct
dk	defined and then killed	Not invalid but probably a programming error
ud	used and defined	Acceptable
uu	used and used again	Acceptable
uk	used and killed	Acceptable
kd	killed and defined	Acceptable
ku	killed and used	A serious defect
kk	killed and killed	Probably a programming error

Data Flow Graphs

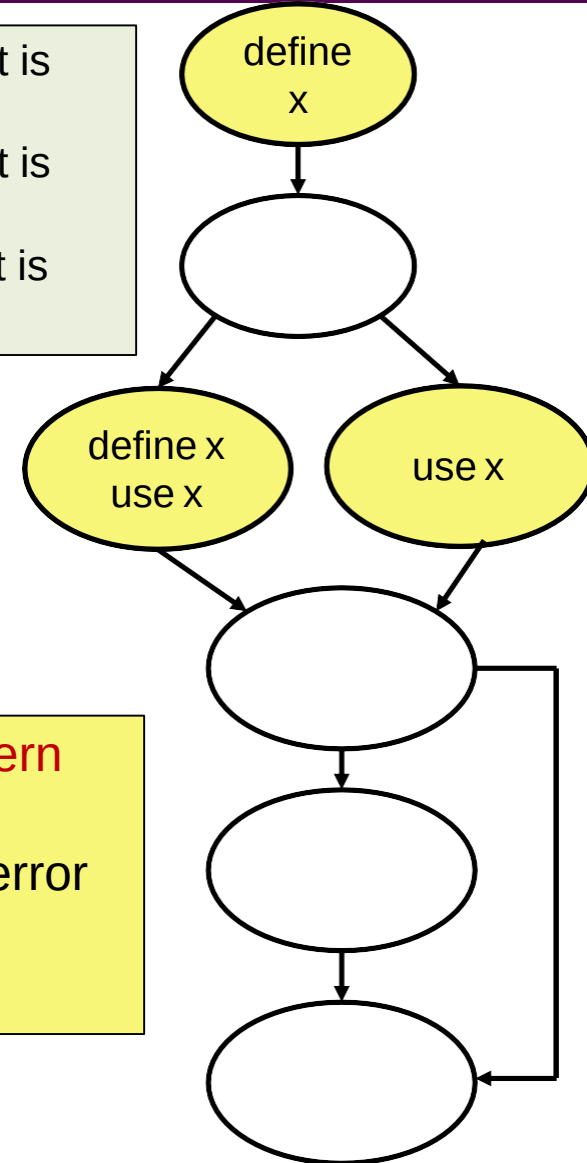


- $\sim d$: the variable does not exist, then it is defined
- $\sim u$: the variable does not exist, then it is used
- $\sim k$: the variable does not exist, then it is killed

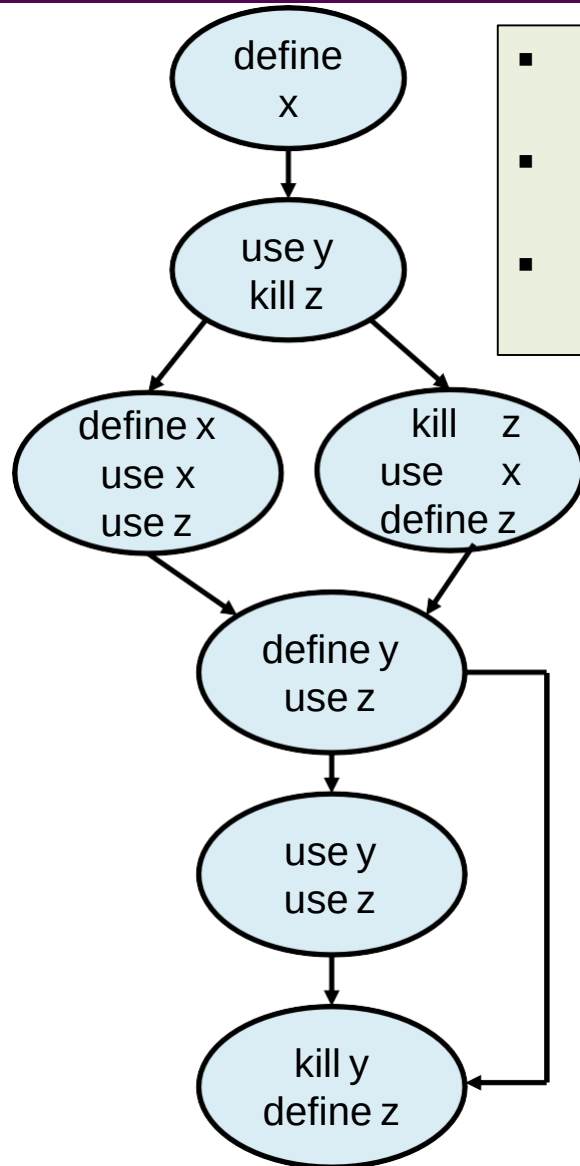
Control flow graph annotated with define-use-kill information for x, y, z

Variable x: define-use-kill pattern
 $\sim d$: correct
dd: suspicious, programming error
du: correct
du: correct

Variable x



Data Flow Graphs



- $\sim d$: the variable does not exist, then it is defined
- $\sim u$: the variable does not exist, then it is used
- $\sim k$: the variable does not exist, then it is killed

Control flow graph annotated with define-use-kill information for x, y, z

Variable y:

 $\sim u$: major blunder

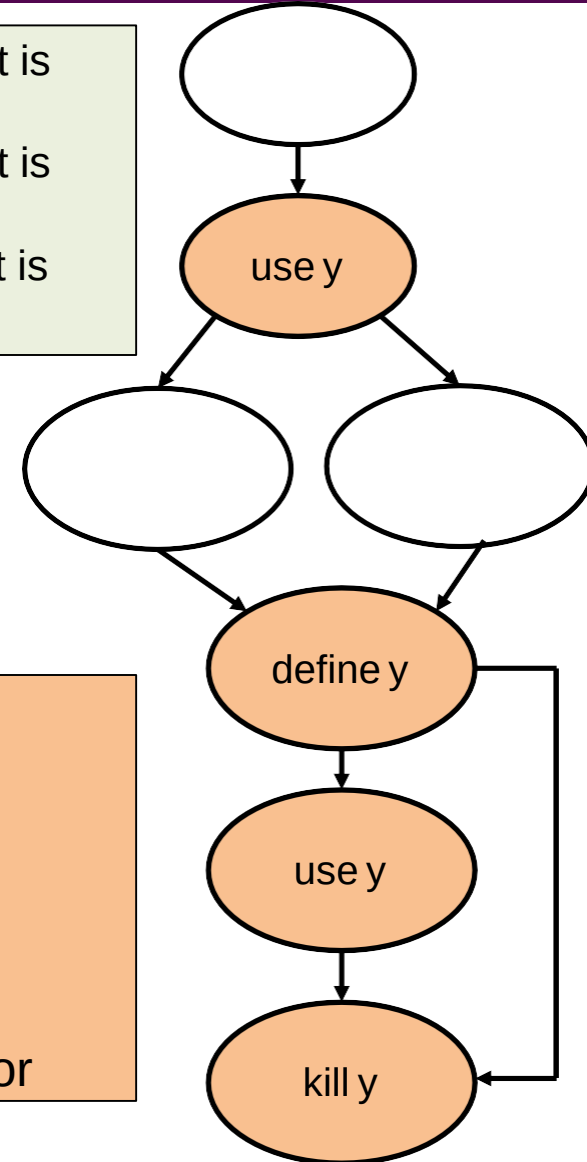
ud: acceptable

du: correct

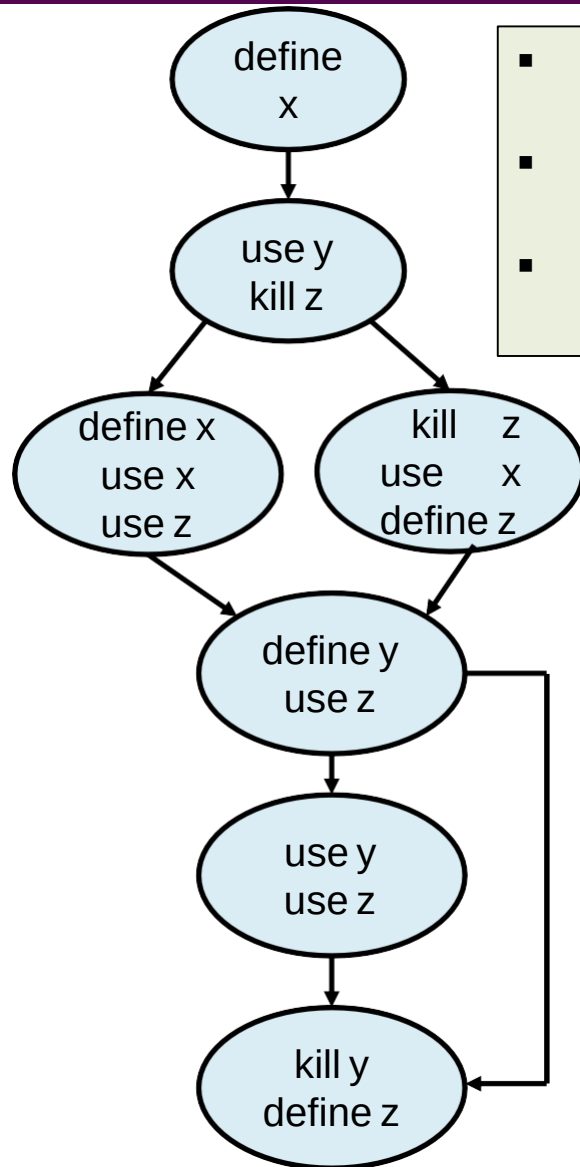
uk: acceptable

dk: probable programming error

Variable y



Data Flow Graphs



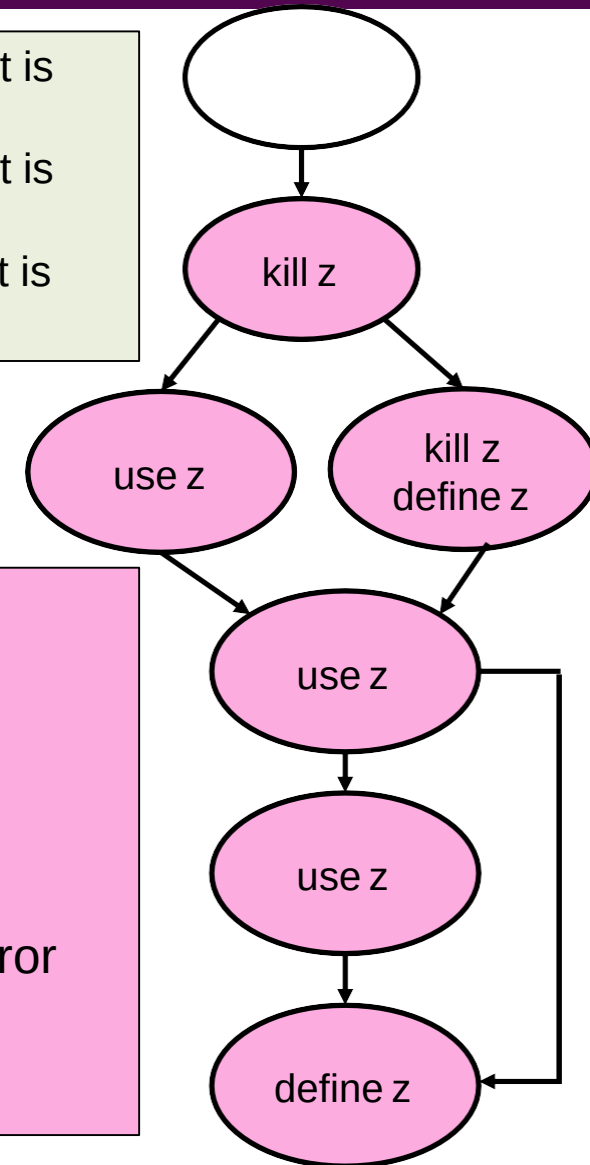
- $\sim d$: the variable does not exist, then it is defined
- $\sim u$: the variable does not exist, then it is used
- $\sim k$: the variable does not exist, then it is killed

Control flow graph annotated with define-use-kill information for x, y, z

Variable z

 $\sim k$: programming error
 ku : major blunder
 uu : correct
 ud : acceptable
 kk : probable programming error
 kd : acceptable
 du : correct

Variable z



Slice-Based Testing

- **Program slice $S(V, n)$:** Given a program P and a set V of variables in P , a **slice** on the variable set V at statement n is the set of all statements in P that contribute to the values of variables in V
- The idea of slices is to separate a program into components that have some useful (functional) meaning

Program slice on variable i

```
1. s = 0;  
2. i = 1;  
3. while (i <= n)  
   {  
4.     s += i;  
5.     i ++  
   }  
6. cout << s;  
7. cout << i;  
8. cout << n;
```

Program slice on variable i

```
1. s = 0;  
2. i = 1;  
3. while (i <= n)  
    {  
4.     s += i;  
5.     i ++  
    }  
6. cout << s;  
7. cout << i;  
8. cout << n;
```

```
2. i = 1;  
3. while (i <= n)  
    {  
  
5.     i ++  
    }  
  
7. cout << i;
```

Program slice on variable i

```
1. s = 0;  
2. i = 1;  
3. while (i <= n)  
    {  
4.     s += i;  
5.     i ++  
    }  
6. cout << s;  
7. cout << i;  
8. cout << n;
```

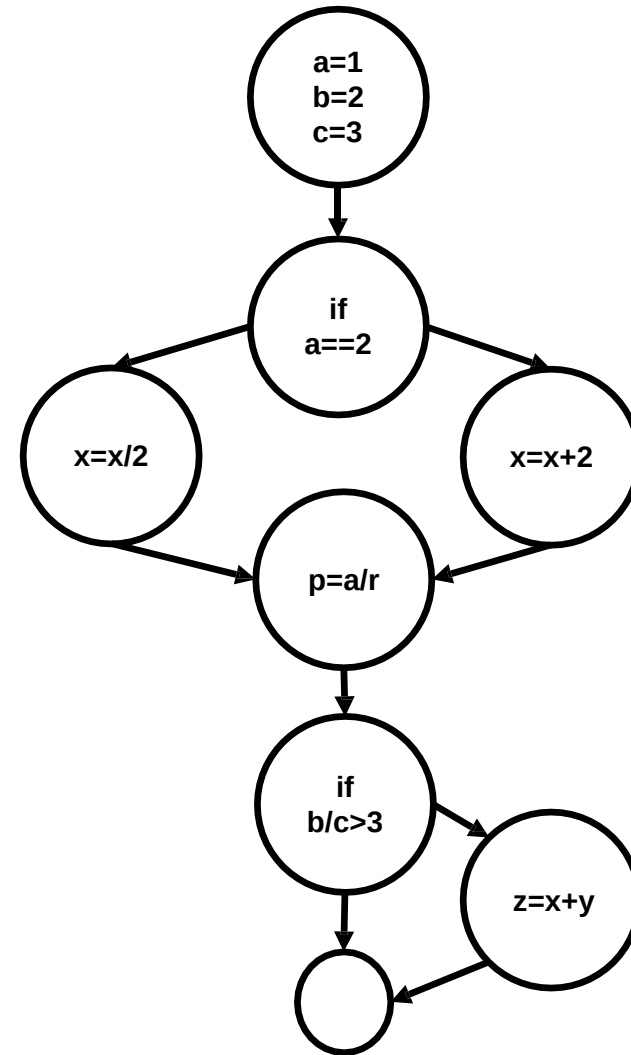
```
def-1 (s)  
def-2 (i)  
    use-3 (i)* --loop  
    use-3 (n)*--loop  
        def-4 (s)  
        use-4 (s)  
        use-4 (i)  
        def-5 (i)  
        use-5 (i)  
  
use-6 (s)  
use-7 (i)  
use-8 (n)
```

Applicability and Limitations

- + It should be **used** for **all modules** of **code** that **cannot be tested sufficiently** through reviews and inspections
- Tester must have **sufficient programming** skill
- Can be very **time consuming**

Task

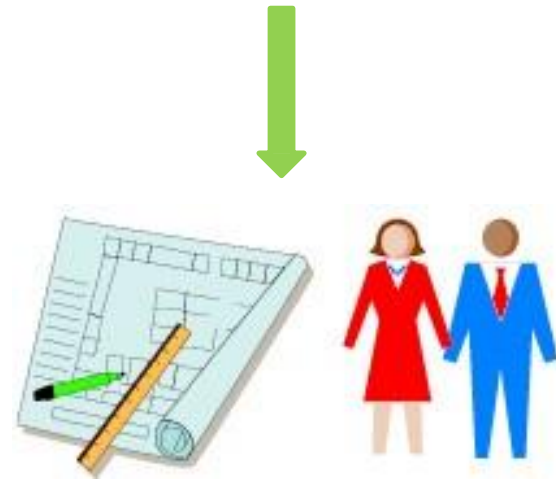
```
a = 1;  
b = 2;  
c = 3;  
if (a==2) {x = x / 2;}  
else {x = x + 2;}  
p = a/r;  
if (b/c > 3) {z = x + y;}
```



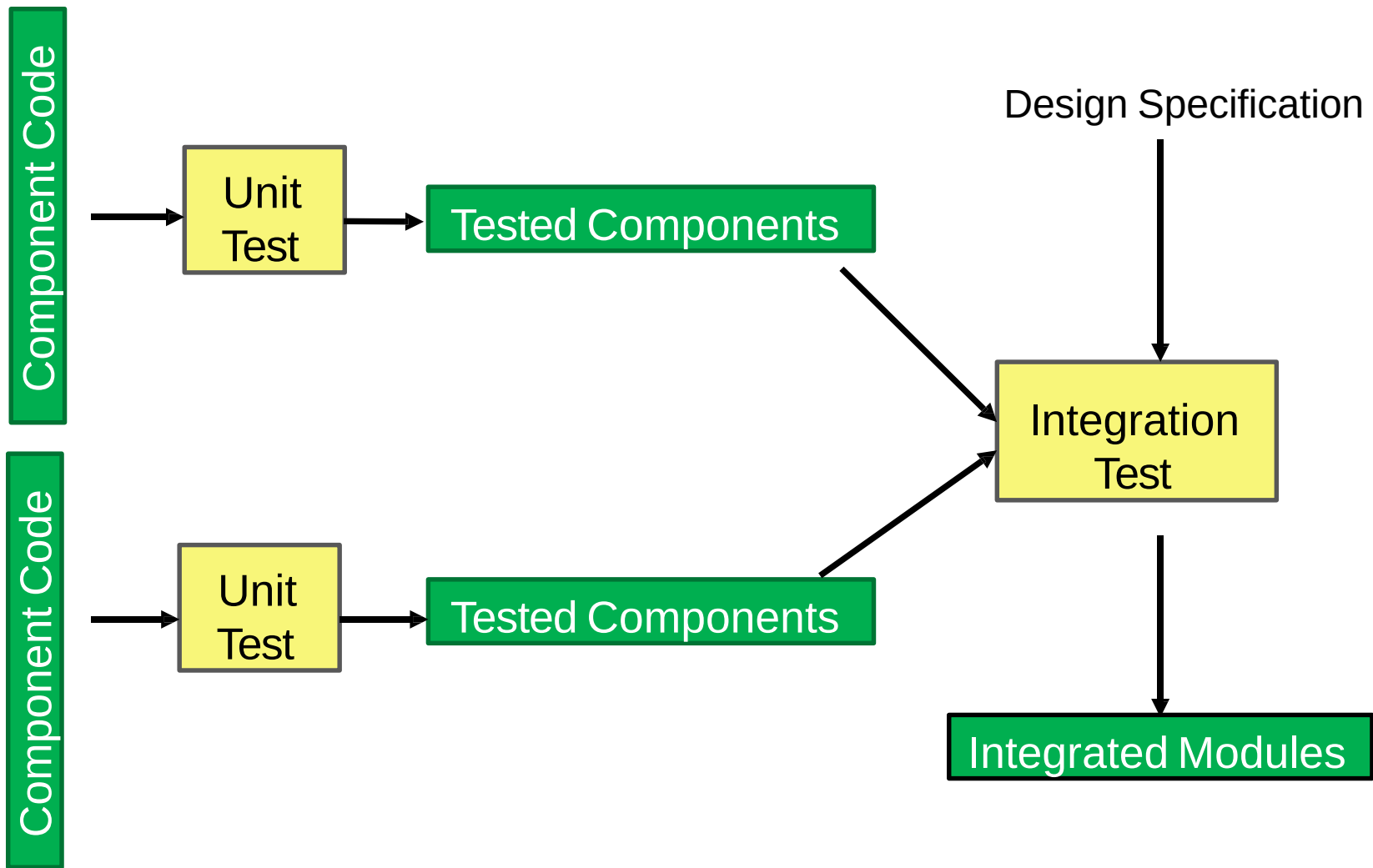
Integration Testing

Unit & Integration Testing

- **Objective:** To ensure that code implemented the design properly

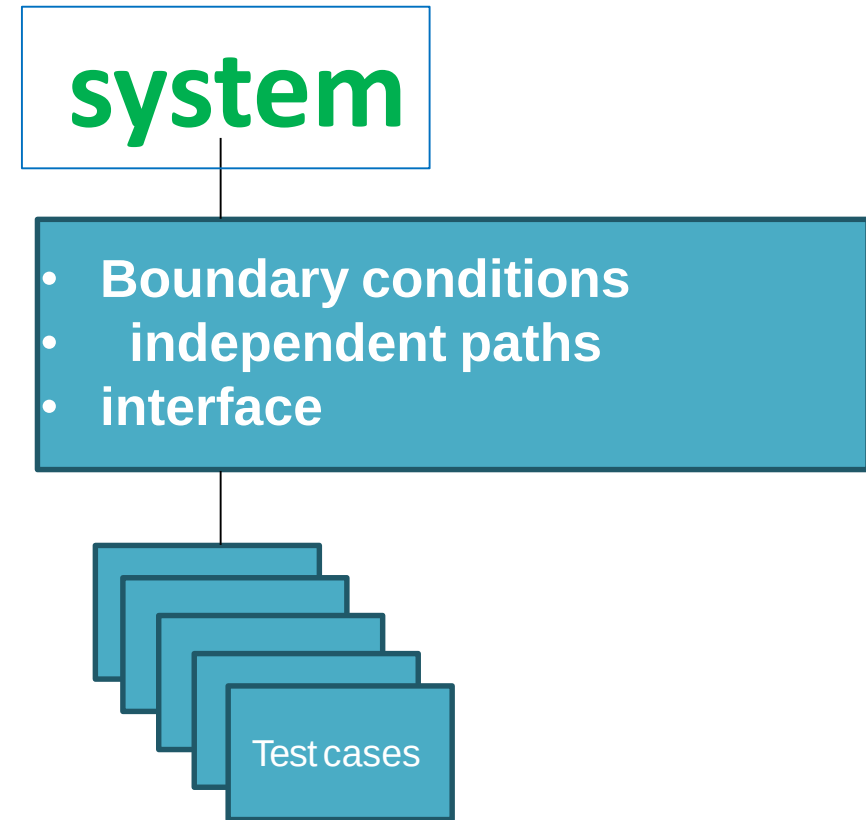
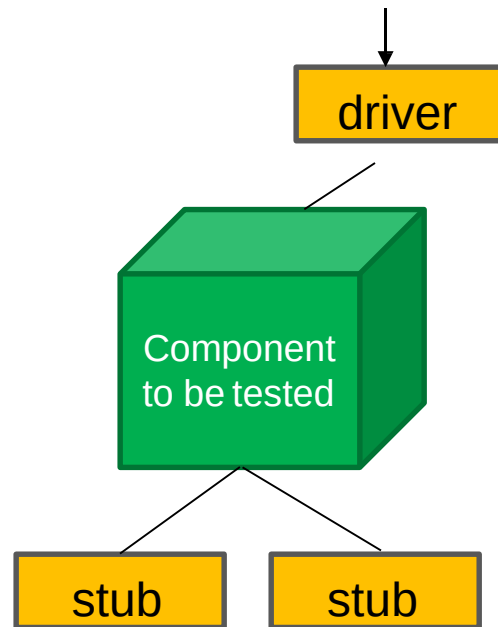


Design Specification



Stubs and Drivers

- **Stubs:** **throw-away** piece of code that takes place of the actual code. It emulates a called unit
- **Drivers:** **throw-away** code that emulate the procedure calls



Suppose you have a function (Function A) that calculates the total marks obtained by a student in a particular academic year. Suppose this function derives its values from another function (Function b) which calculates the marks obtained in a particular subject.

You have finished working on Function A and wants to test it. But the problem you face here is that you can't seem to run the Function A without input from Function B; Function B is still under development. In this case, you create a dummy function to act in place of Function B to test your function. This dummy function gets called by another function. Such a dummy is called a Stub.

To understand what a driver is, suppose you have finished Function B and is waiting for Function A to be developed. In this case you create a dummy to call the Function B. This dummy is called the driver.

Purpose of Integration Testing

- Presumes previously tested units
- Tests functionality "**between**" *unit* and *system* levels
- Emphasis shifts from “how to test” to “**what to test**”

1. Based on Functional Decomposition

- Top-Down
- Bottom-up
- Sandwich
- Big Bang

2. Based on Call Graph

- Pair-Wise
- Neighborhood

3. Based on Paths

- MM-Paths



Decomposition-Based Integration

- Top-Down
- Bottom-up
- Sandwich
- Big Bang

All these integration orders presume that the **units** have been **separately tested**, thus, the **goal** of decomposition-based integration is to **test the interfaces** among separately tested units.

- In this view of integration, **all the units are compiled together and tested at once**

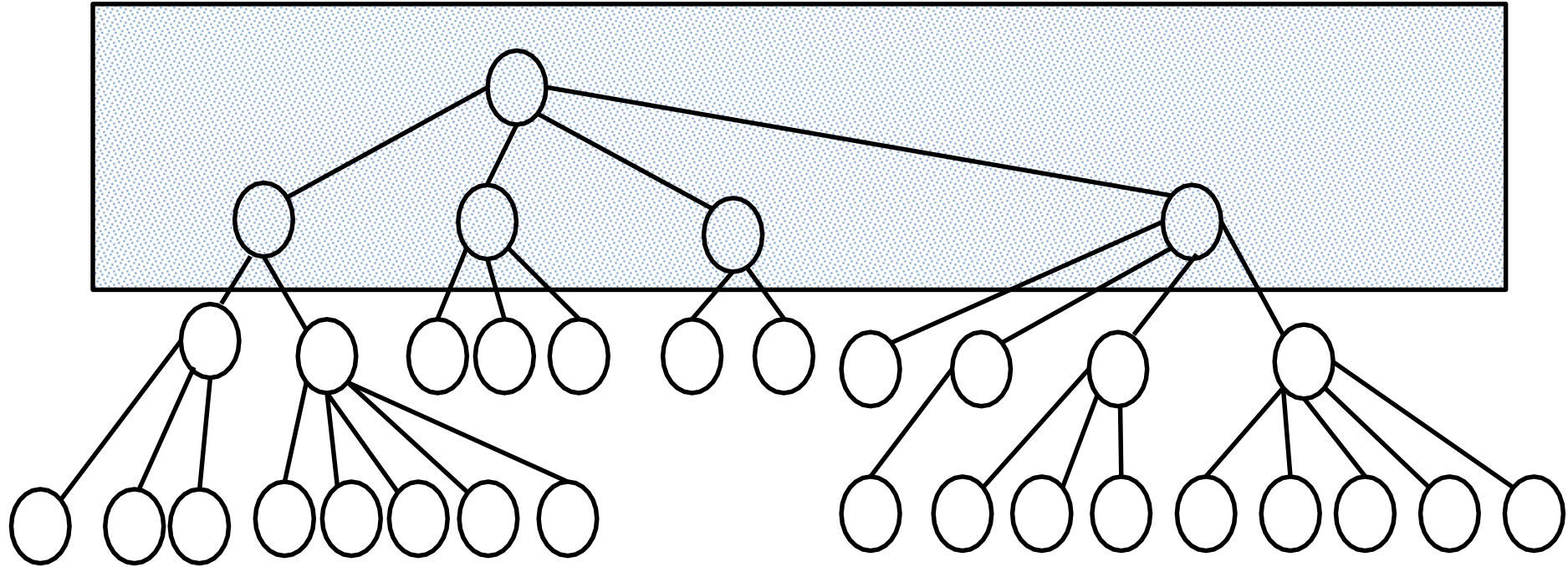
Execute Test Cases

- The **draw back** to this is that when a **failure** is observed, **few clues** are available to help isolate the **location(s) of the fault**

Top-Down Integration

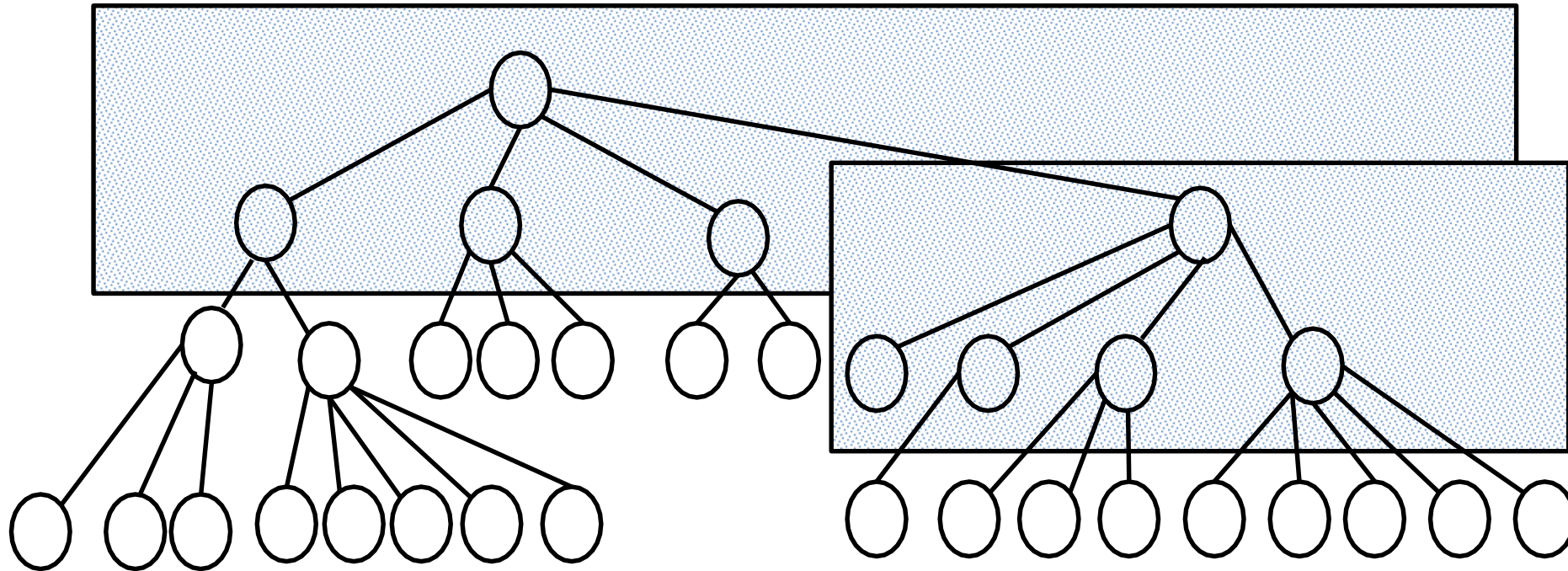
Top-Down Integration

Top Subtree



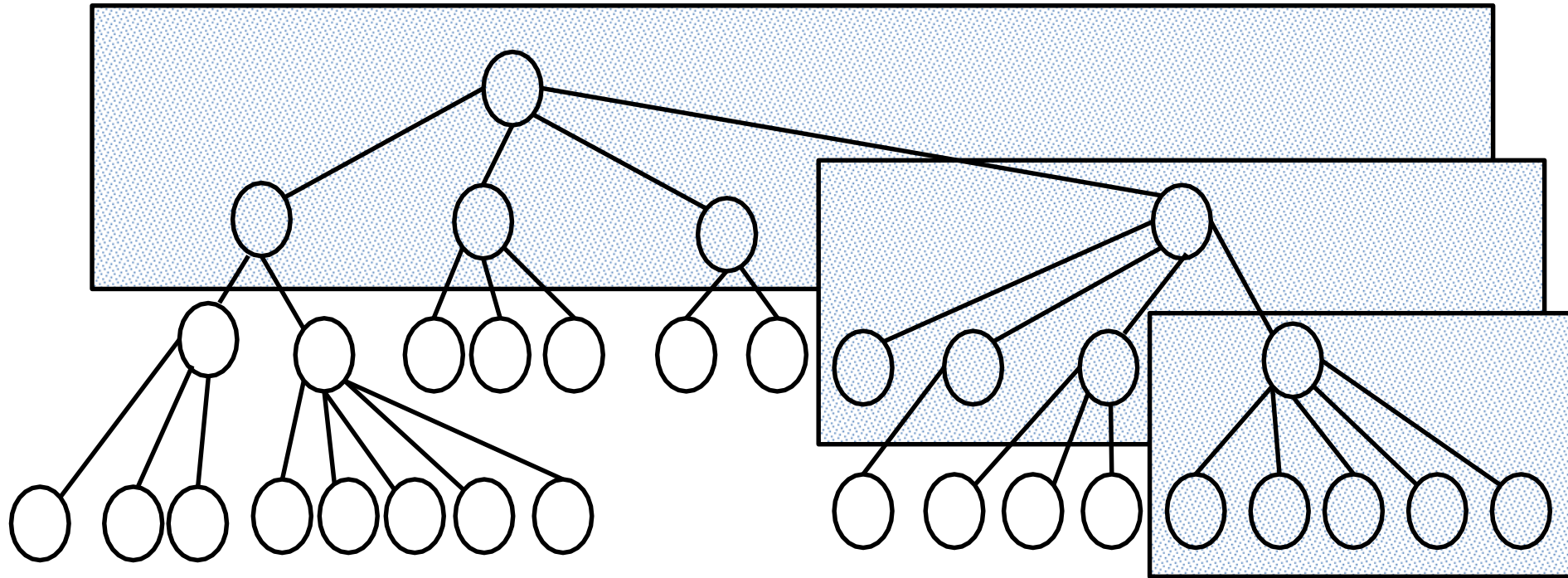
Top-Down Integration

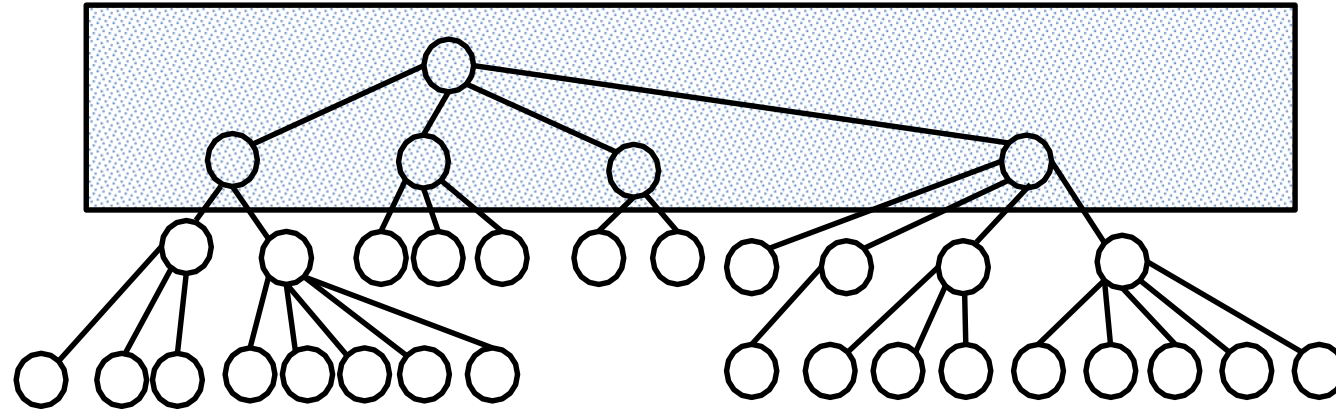
Second Level Subtree



Top-Down Integration

Bottom Level Subtree



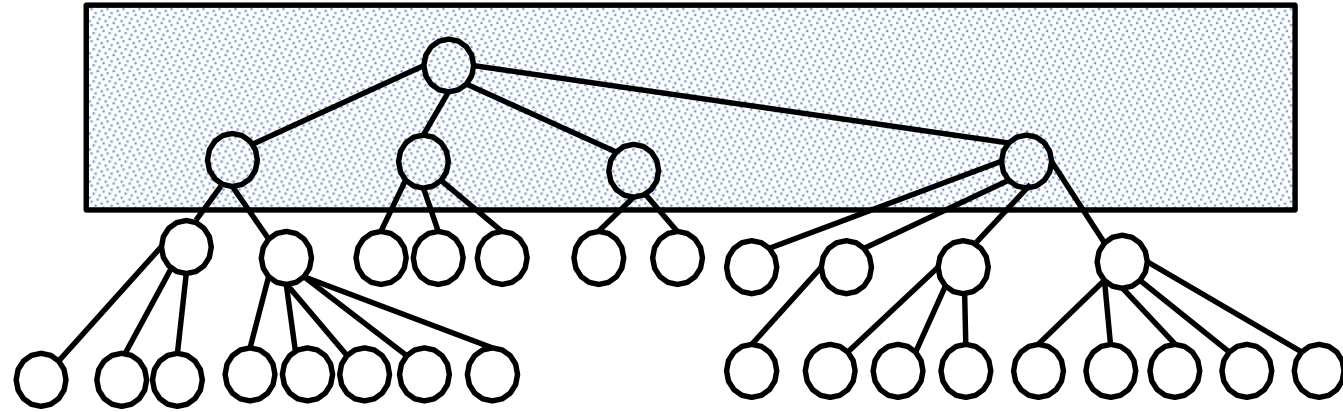


Top Subtree

- x stubs
- y integration sessions

- A test session is one set of tests for a specific configuration **actual code** and **stubs**

- In each, one component would be actual code, and the other three would be stubs

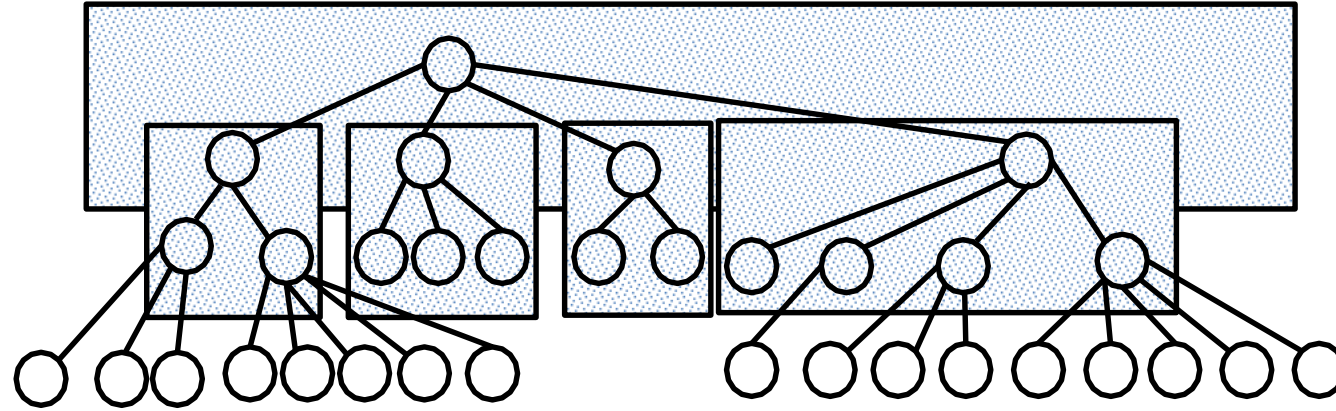


Top Subtree

- 4 stubs
- 5 integration sessions
- A test session is one set of tests for a specific configuration actual code and stubs

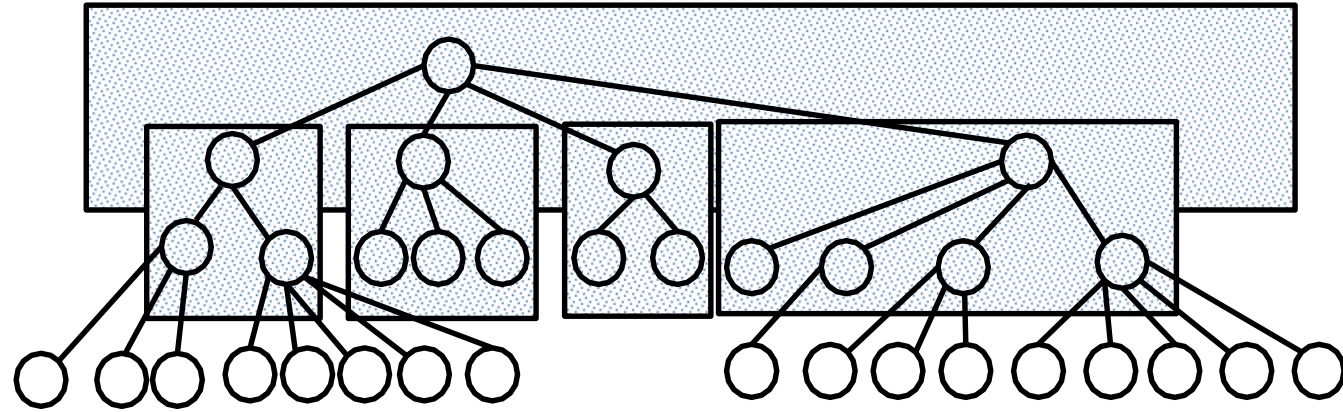
In each, one component would be actual code, and the other three would be stubs

Top-Down Integration



Second Level Subtree

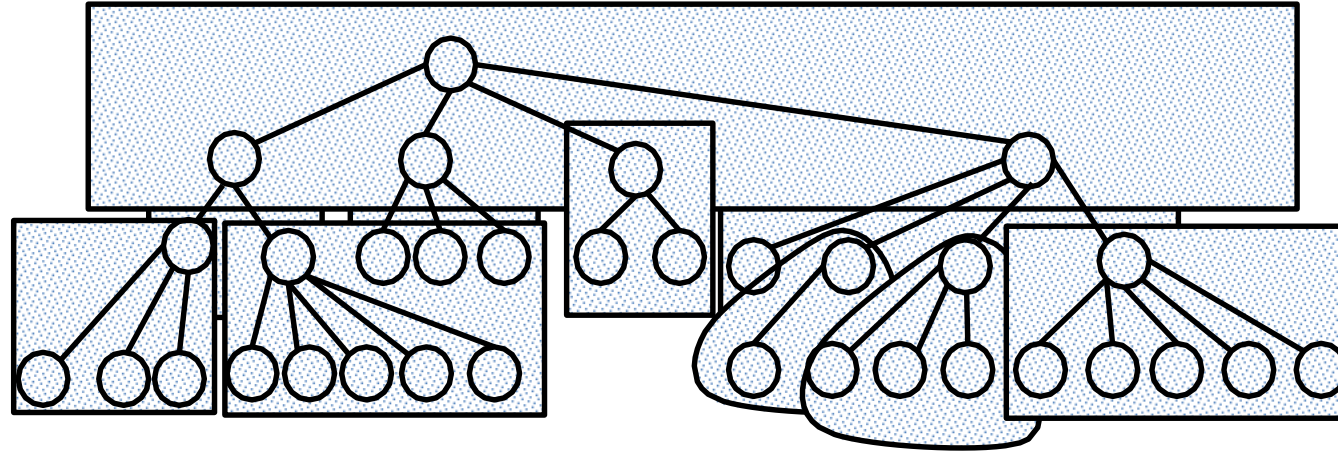
- x stubs
- y integration sessions



Second Level Subtree

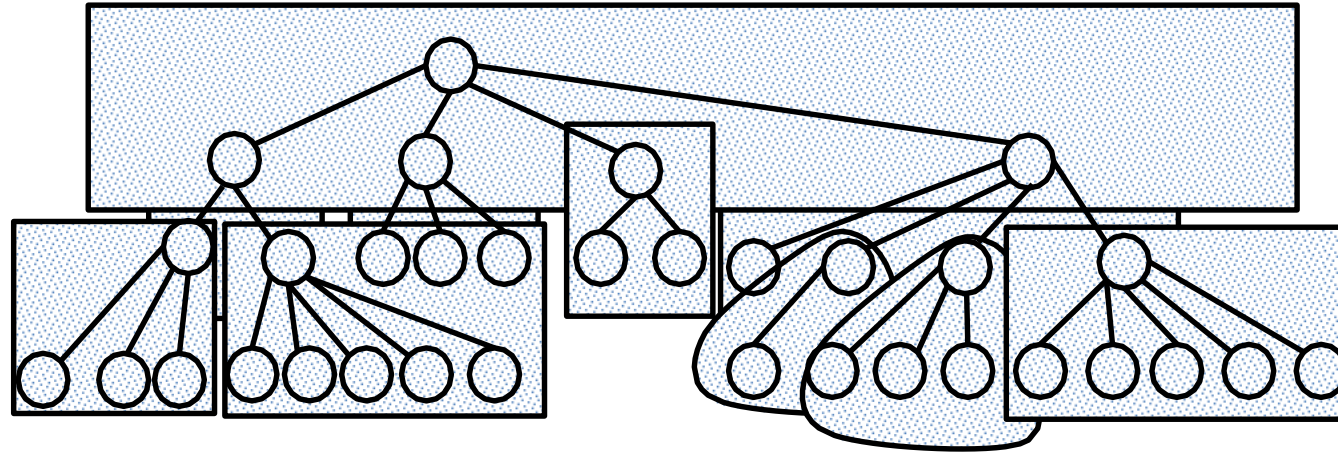
- 15 stubs ($3+4+3+5$)
- 20 integration sessions ($15 \text{ stubs} + 4 \text{ roots (subtrees)} + 1$)

Top-Down Integration



Bottom Level Subtree

- x stubs
- y integration sessions



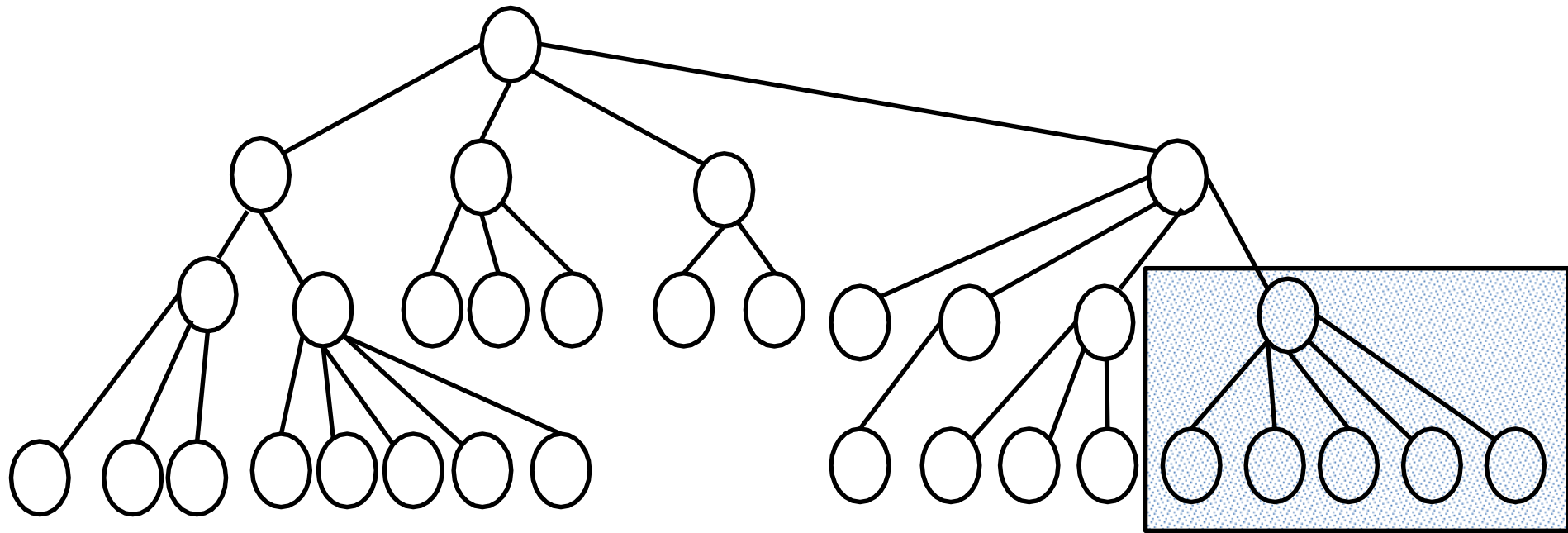
Bottom Level Subtree

- 32 stubs
- 42 integration sessions (32 stubs + 9 roots (subtrees) + 1 (with all the stubs))

Bottom-Up Integration

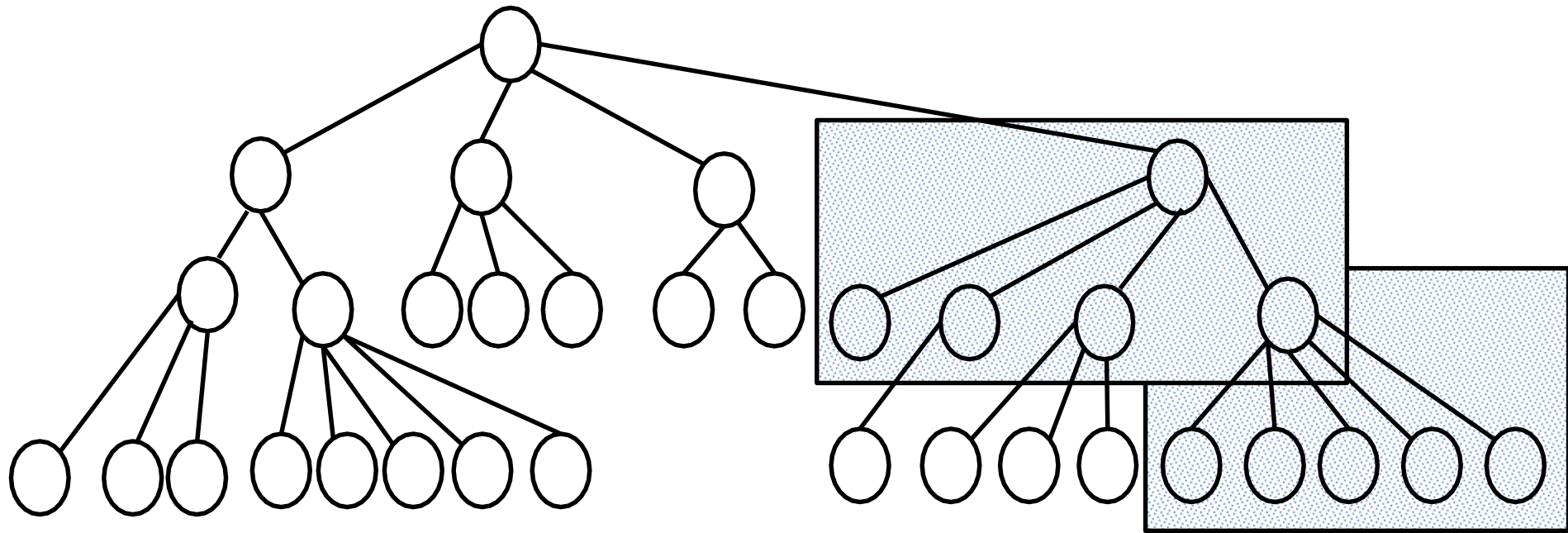
Bottom-Up Integration

Bottom Level Subtree



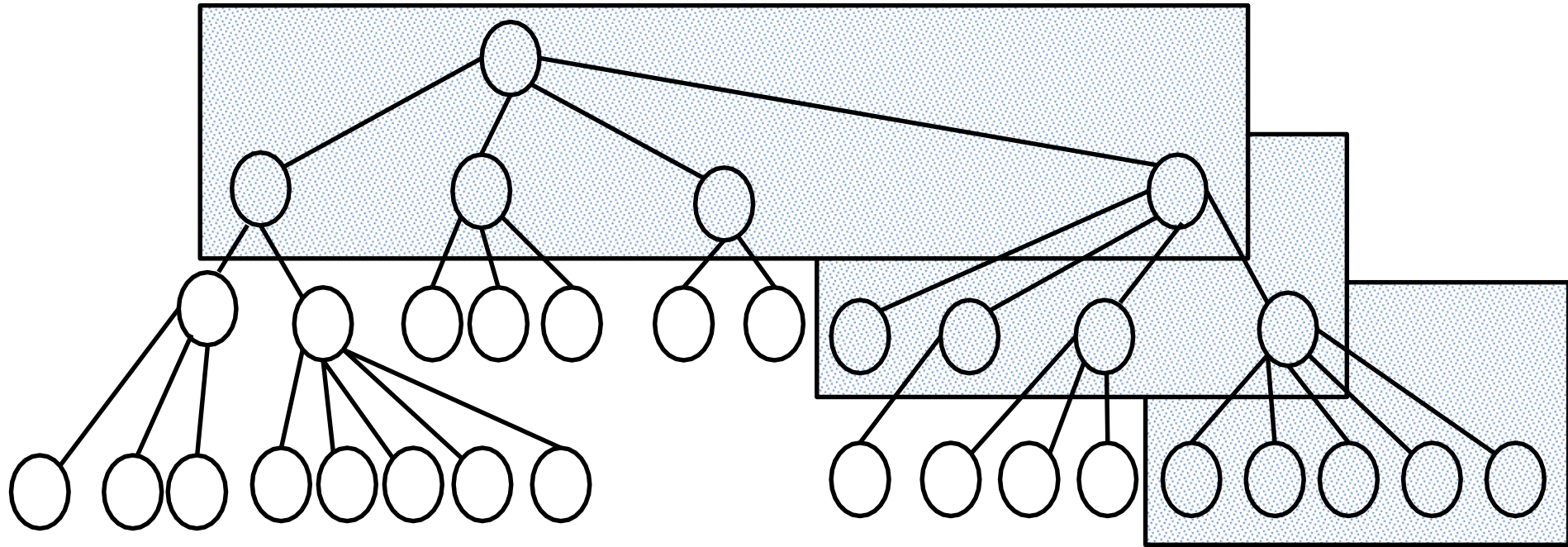
Bottom-Up Integration

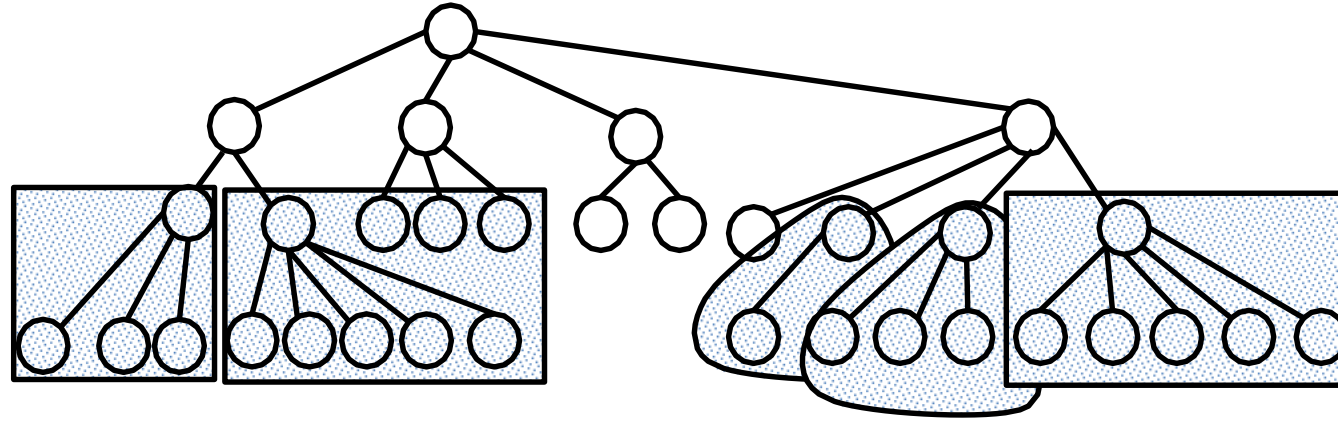
Second Level Subtree



Bottom-Up Integration

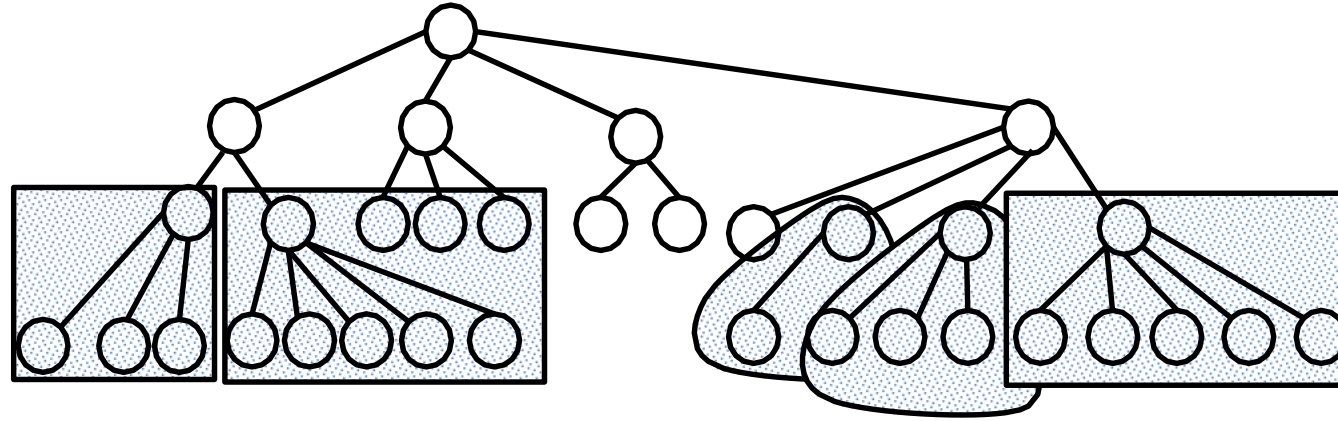
Top Level Subtree





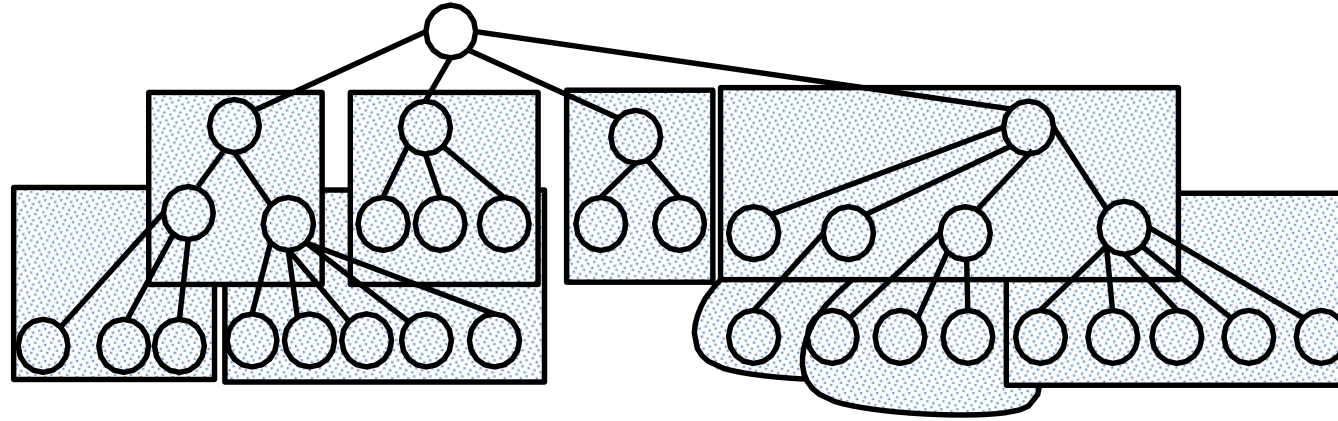
Bottom Level Subtree

- **x** drivers
- **y** integration sessions



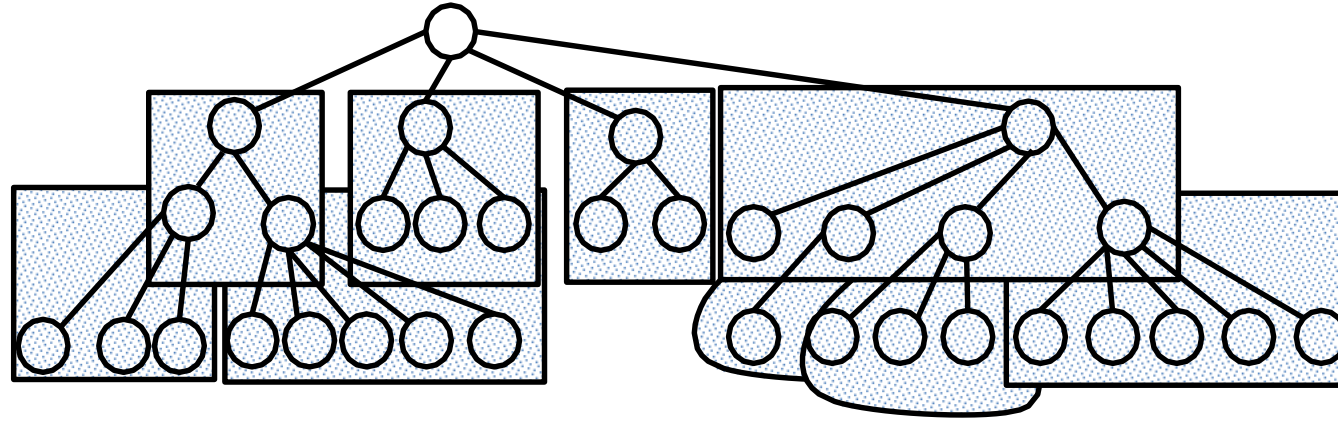
Bottom Level Subtree

- 5 drivers (5 roots (subtrees))
- 17 integration sessions (3+5+1+3+5)



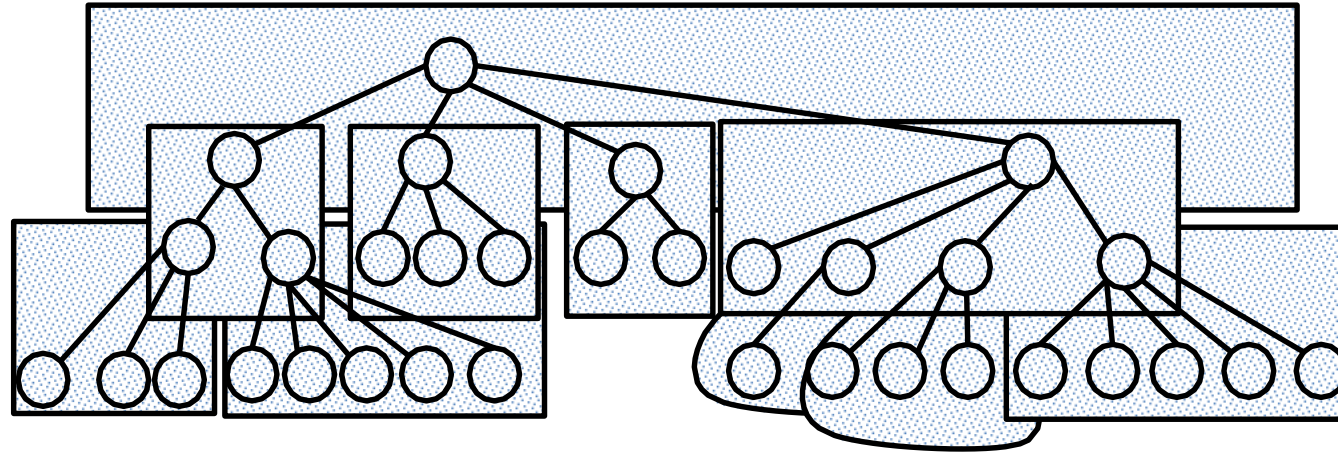
Second Level Subtree

- **x** drivers
- **y** integration sessions



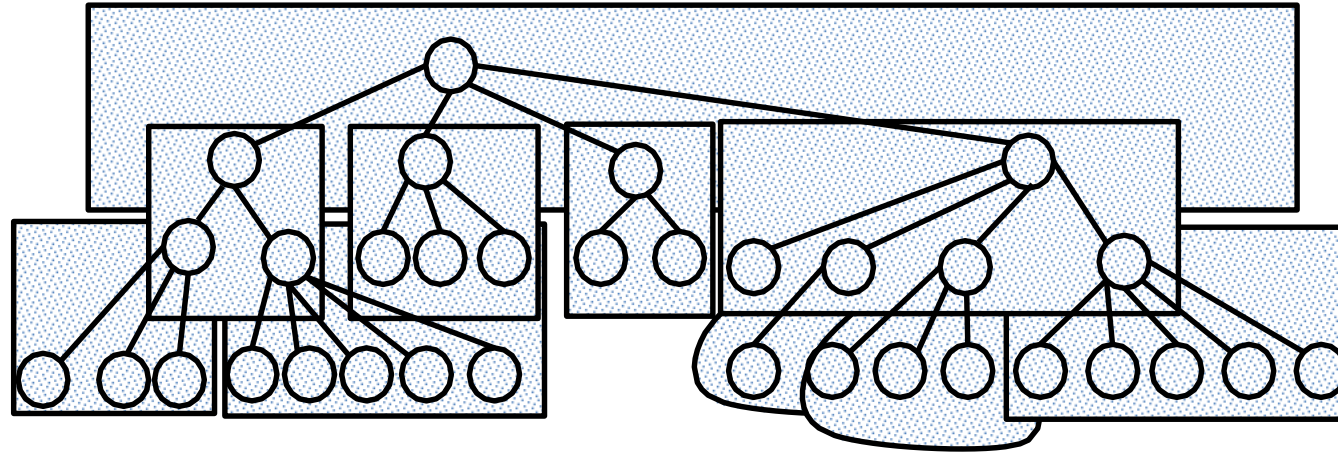
Second Level Subtree

- 9 drivers (9 subtrees)
- 28 integration sessions (17+2+3+2+4)



Top Level Subtree

- **x** drivers
- **y** integration sessions



Top Level Subtree

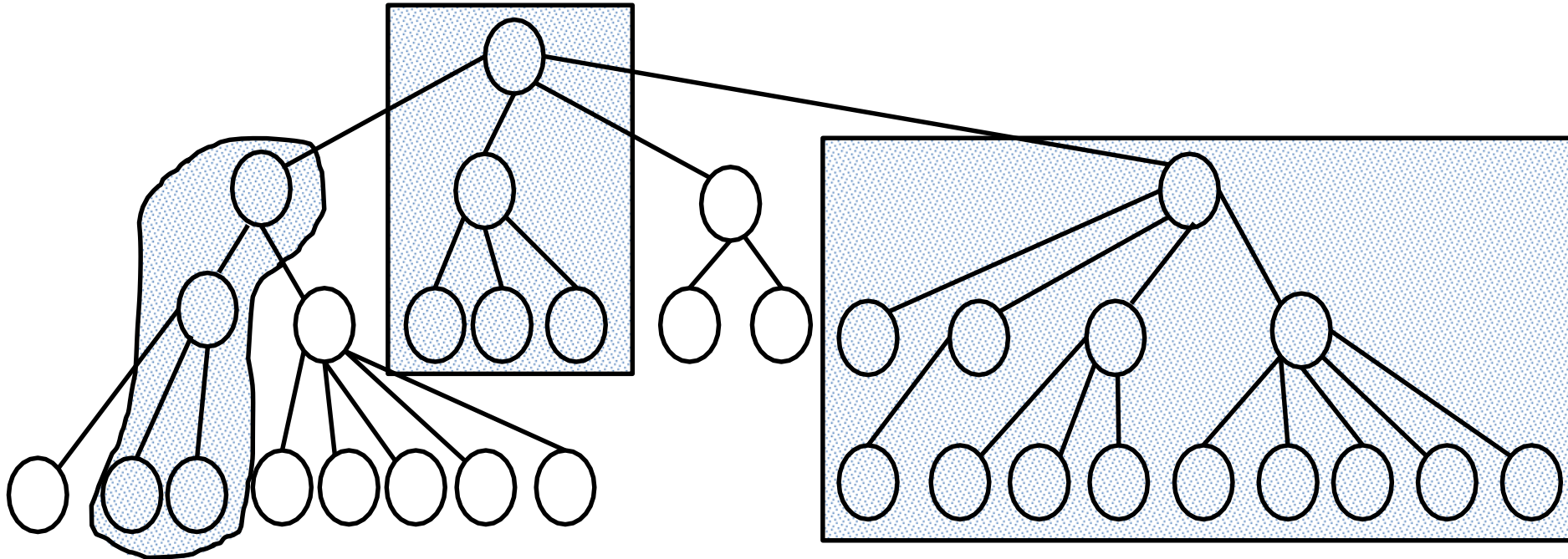
- 10 drivers (9 subtrees + 1 (main))
- 42 integration sessions (28+4+ 9 roots (subtrees) + 1 (with all the drives))

- **Sessions** = Nodes – Leaves + Edges
- **For top-down integration:** (nodes -1) stubs are needed
- **For bottom-up integration:** (nodes -leaves) drivers are needed

Sandwich Integration

- A combination of top-down and bottom-up integration
- Less stub and drivers development effort
- Added difficulty to fault isolation

Sandwich Integration



Decomposition-Based Integration - Pros & Cons

- Whenever a failure is observed, the **most recently** added **unit** is suspected.
- The top-down and bottom-up terms suggest breadth-first traversals of the decomposition tree, but this is not mandatory.
- The **developing effort** for **stubs** or **drivers** is one drawback to these approaches, and this is compounded by the **retesting effort**.

- One of the most frequent objections to functional decomposition and waterfall development is that both are **artificial**, and both serve the **needs** of **project management** more than the **needs** of **software developers**. This holds true also for decomposition-based testing. The whole mechanism is that **units are integrated** with respect to **structure**; this presumes that **correct behavior** follows from **individually correct units** and **correct interfaces**.

Recap

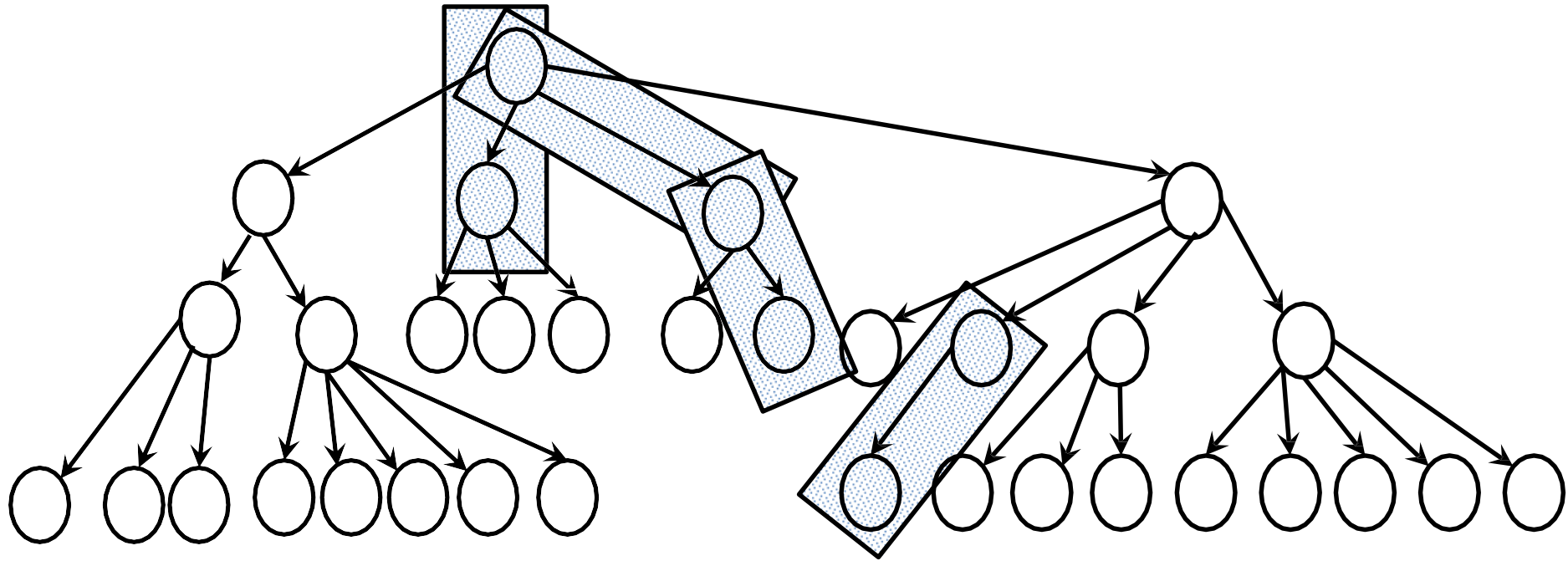
- Integration Testing
- Based on Functional Decomposition
 - Top-Down
 - Bottom-up
 - Sandwich
 - Big Bang

Call Graph-Based Integration

- One of the **drawbacks** of decomposition based integration is that the **basis** is the **functional decomposition tree**
- Call graph **mitigate** this deficiency
- **Behavioral** testing
- Two approaches to integration testing
 - **Pair-wise** integration
 - **Neighborhood** integration

- The idea behind pair-wise integration is to eliminate the stub/driver development effort
- Similar to top-down or bottom-up integration
- Uses **real code** instead of stubs / drivers
- **Reduces testing effort** (no need to write stubs/drivers)

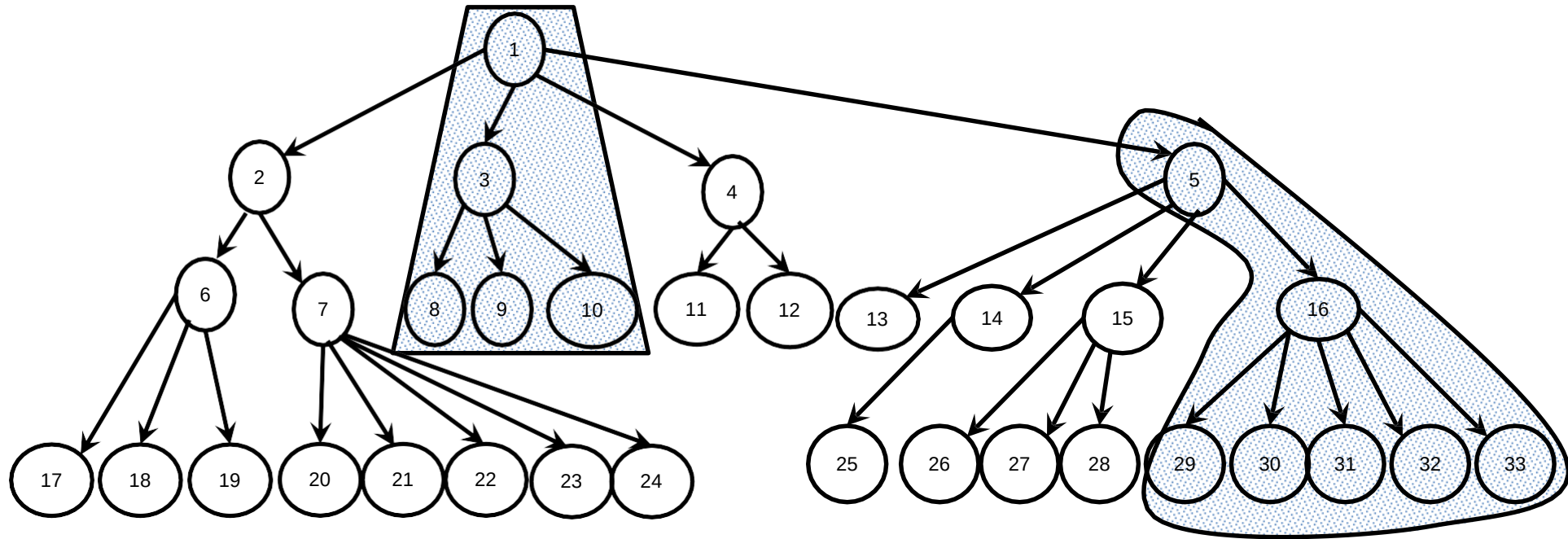
Pair-Wise Integration



- Typical top-down or bottom-up strategy testing selects nodes in the **breadth-first order**
- **Neighbourhood** integration selects nodes in the **depth-first order**
- Avoids the need to write stubs / drivers

- The neighborhood of a node in a graph is the set of nodes that are one edge away from the given node
- In a directed graph, this includes all the immediate predecessor nodes and all the immediate successor nodes (these correspond to the set of stubs and drivers of the node)

Neighbourhood Integration



Node	Predecessors	Successors
....		
3	1	8, 9, 10
16	5	29, 30, 31, 32, 33
...	...	...
...	...	...
...	...	...
...	...	...

- Source node: a node with **indegree**= 0
- Sink node: a node with **outdegree**= 0

▪ Interior nodes = nodes-(source nodes + sink nodes)

Neighborhoods = interior nodes + source nodes

Neighborhoods = nodes – (source nodes + sink nodes) + source nodes

Neighborhoods = nodes – sink nodes

- + **Eliminate** the **stub/driver** development effort.
- + Matches well with **developments** characterized by **build** and **composition**
- The biggest drawback is the **fault isolation problem**, specially for large neighborhood. Resolving a fault in a node means **changing the unit's code** which in turn means that all previously tested neighborhoods that contain the changed node need to be tested

Path-Based Integration

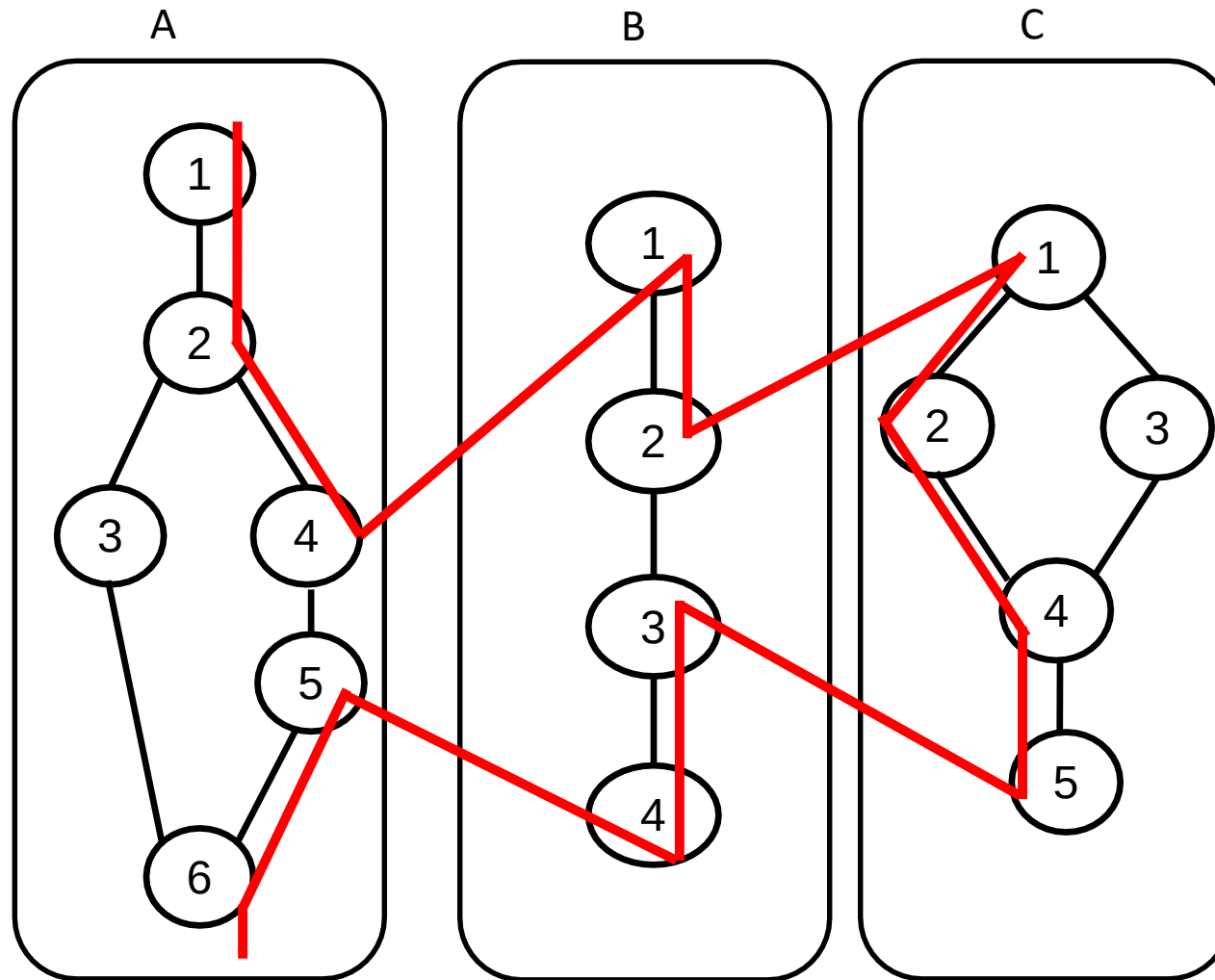
- A **source node** in a program is a statement fragment at which program execution begins or resumes
- A **sink node** in a program is a statement fragment at which program execution terminates
- A **Module Execution Path (MEP)** is a sequence of statements that begins with a source node and ends with a sink node

- A **message** is a programming language mechanism by which one unit transfers control to another unit – Examples: procedure call, function references

- An **MM-path** is an interleaved sequence of module execution paths and messages

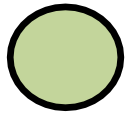
- Given a set of units, their MM-Path graph is the directed graph in which **nodes** are **module execution paths** and **edges** correspond to **messages** and **returns** from one unit to another

MM Paths

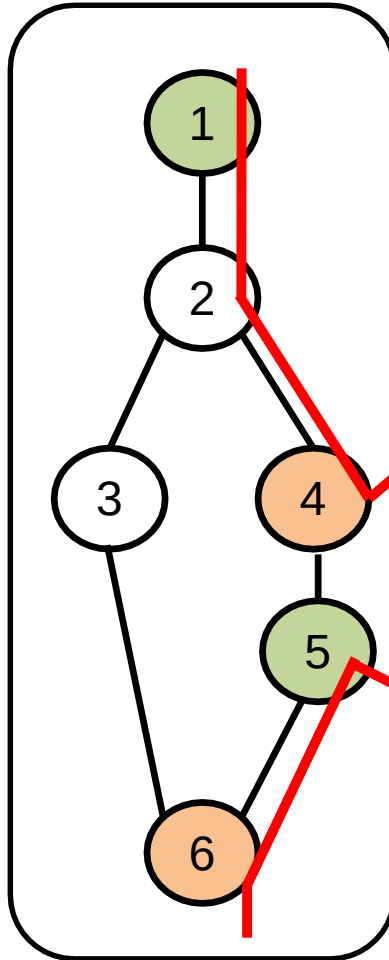
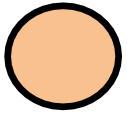


A **sink node** in a program is a statement fragment at which program execution terminates

Source Nodes



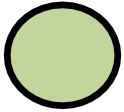
Sink Nodes



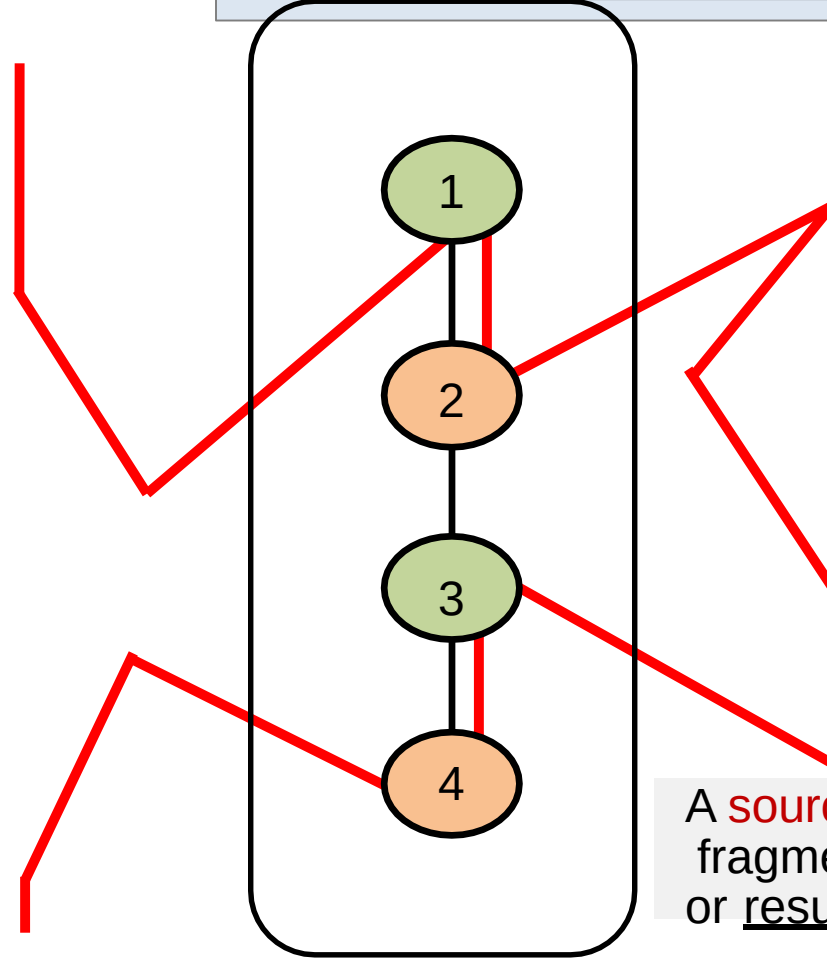
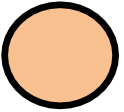
A **source node** in a program is a statement fragment at which program execution begins or resumes

A **sink node** in a program is a statement fragment at which program execution terminates

Source Nodes



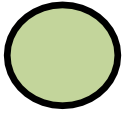
Sink Nodes



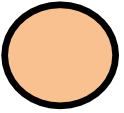
A **source node** in a program is a statement fragment at which program execution begins or resumes

A **sink node** in a program is a statement fragment at which program execution terminates

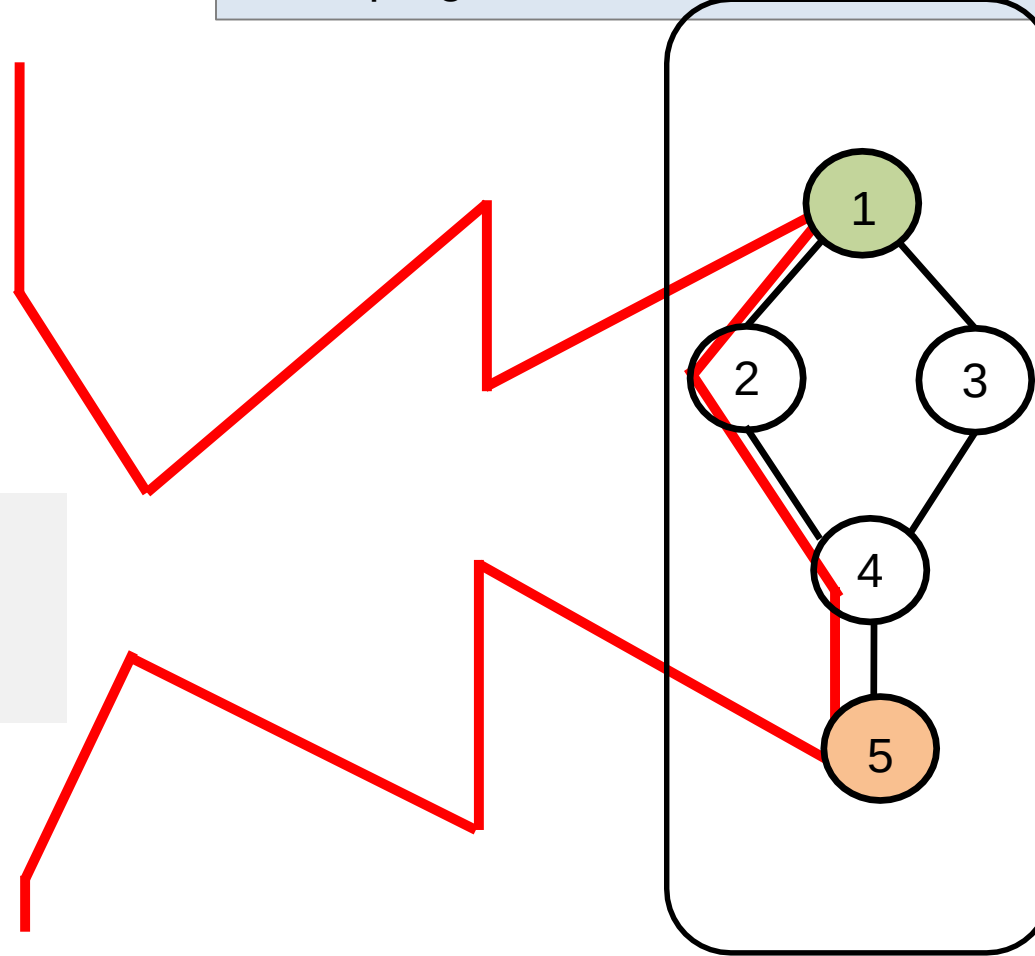
Source Nodes



Sink Nodes

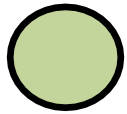


A **source node** in a program is a statement fragment at which program execution begins or resumes

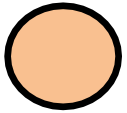


A **sink node** in a program is a statement fragment at which program execution terminates

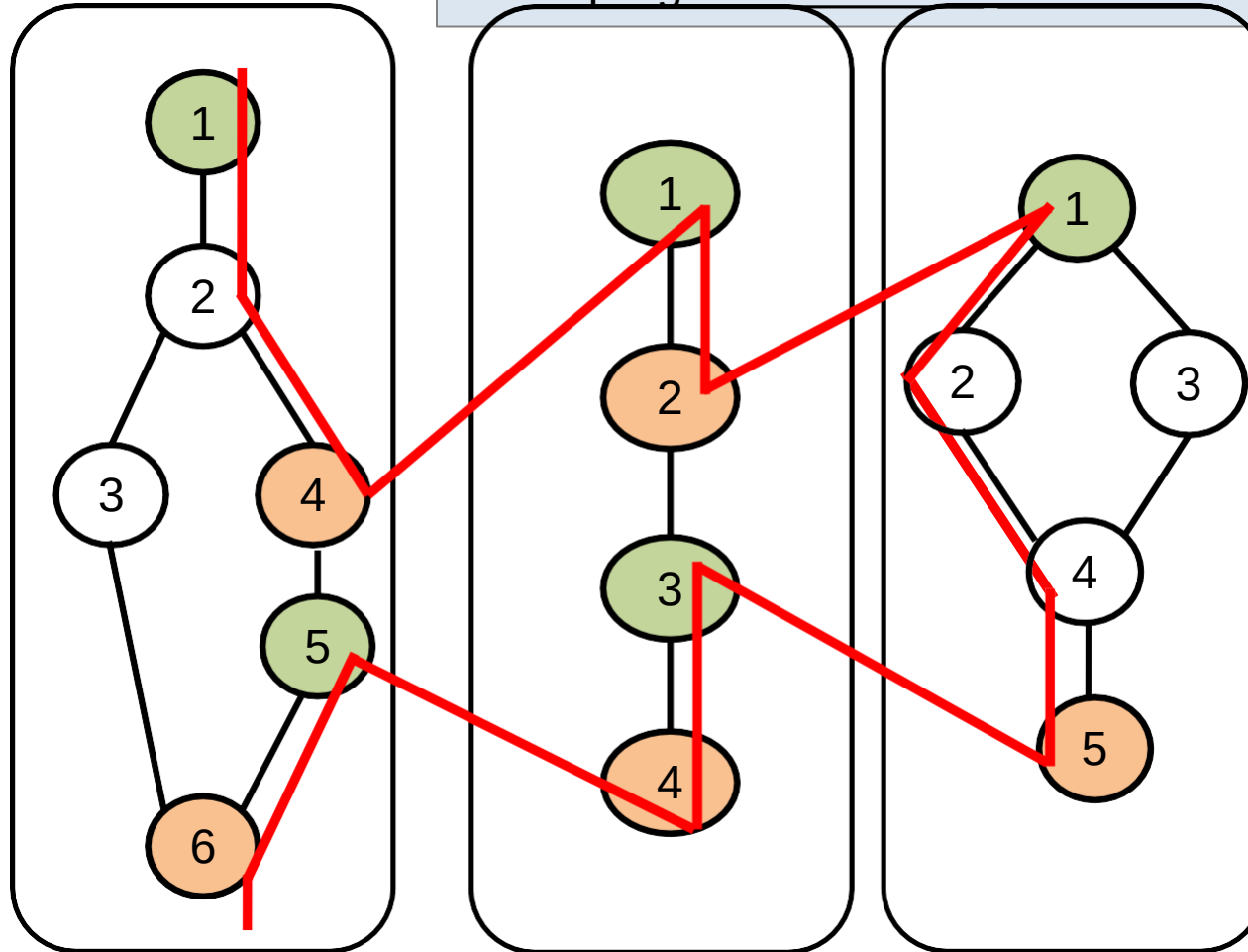
Source Nodes



Sink Nodes



A **source node** in a program is a statement fragment at which program execution begins or resumes



■ MEP (A, I) = < 1, 2, 3, 6 >

■ MEP (A, II) = < 1, 2, 4 >

■

■

■

■

■

- $\text{MEP (A, I)} = \langle 1, 2, 3, 6 \rangle$

- $\text{MEP (A, II)} = \langle 1, 2, 4 \rangle$

- $\text{MEP (A, III)} = \langle 5, 6 \rangle$

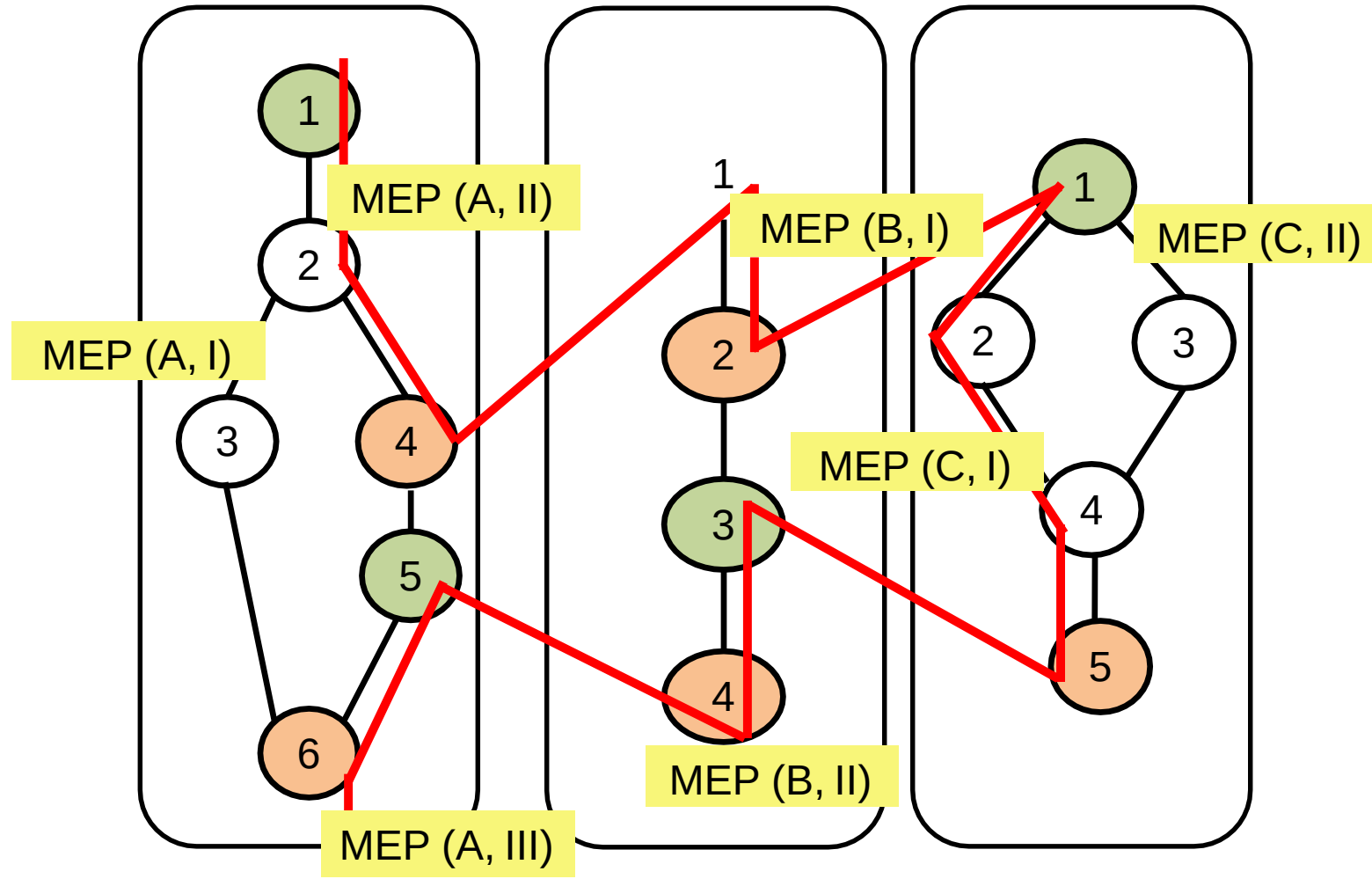
- $\text{MEP (B, I)} = \langle 1, 2 \rangle$

- $\text{MEP (B, II)} = \langle 3, 4 \rangle$

- $\text{MEP (C, I)} = \langle 1, 2, 4, 5 \rangle$

- $\text{MEP (C, II)} = \langle 1, 3, 4, 5 \rangle$

MM Paths



- MM-Paths are a hybrid of **functional** and **structural testing**
- Path-based integration works equally well for software developed in the traditional **waterfall process** or with one of the **composition-based** alternative life cycle models
- Path-based integration testing is **closely coupled** with the **actual system behavior**

- More effort is needed to identify the MM- Paths. This effort is probably offset by the elimination of stub and driver development.